



UNIVERSIDAD DE MÁLAGA



E.T.S. INGENIERÍA
INFORMÁTICA

UNIVERSIDAD DE MÁLAGA

Bachelor's Degree in Computer Engineering

Un orquestador de máquinas virtuales para entornos
docentes y de investigación. Prueba de concepto

A virtual machine orchestrator for teaching and
research environments. Proof of concept

Realizado por

Claudio María Martínez Velasco

Tutorizado por

Dr. Guillermo Pérez Trabado

Departamento

Department of Computer Architecture

Málaga, June 2026

DRAFT



UNIVERSIDAD
DE MÁLAGA



ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA INFORMÁTICA
GRADUATED IN COMPUTER ENGINEERING

**Un orquestador de máquinas virtuales para entornos
docentes y de investigación. Prueba de concepto**

**A virtual machine orchestrator for teaching and
research environments. Proof of concept**

Realizado por

Claudio María Martínez Velasco

Tutorizado por

Dr. Guillermo Pérez Trabado

Departamento

Department of Computer Architecture

UNIVERSIDAD DE MÁLAGA

MÁLAGA, JUNE 2026

Abstract

This Bachelor's Thesis presents a proof-of-concept universal virtual-machine orchestrator aimed at teaching and research environments. The system exposes a common abstraction layer that lets administrators manage virtual machines regardless of the underlying hypervisor. The design integrates, in a single model, a tree of resource descriptors with cascade inheritance, an operation engine based on the Command pattern, and a REST API as the public boundary. The architecture is split into two layers: a generic, reusable core, and a domain-specific application for deploying classroom environments built as a client on top of that core.

The system is delivered as a downloadable Docker container that the administrator installs through a single-line installer, together with a command-line client supporting both an interactive REPL and one-shot invocations. Adaptation to each hypervisor goes through pluggable connectors; this version ships a mock connector that exercises the full model without real infrastructure, providing a foundation for the real VMware and Proxmox connectors planned as the continuation of the work.

Keywords: virtual-machine orchestration, infrastructure as code, teaching environments, cascade inheritance, layered architecture.

DRAFT

Resumen

Este Trabajo Fin de Grado presenta una prueba de concepto de un orquestador universal de máquinas virtuales, orientado a entornos docentes y de investigación. El sistema proporciona una capa de abstracción común que permite gestionar máquinas virtuales con independencia del hipervisor subyacente, integrando en un único modelo un árbol de descriptores con herencia en cascada, un motor de operaciones basado en el patrón Command y una API REST como frontera pública. La arquitectura se diseña en dos capas: un núcleo genérico reutilizable y una aplicación específica para despliegue de asignaturas, materializada como capa cliente sobre el primero.

El sistema se entrega como un contenedor Docker descargable que el administrador despliega con un instalador de una sola línea, junto con un cliente de línea de órdenes que opera tanto en modo interactivo como por instrucciones sueltas. La adaptación a cada hipervisor se realiza a través de conectores; en esta versión se incluye un conector simulado que permite ejercitar el modelo completo sin infraestructura real, sirviendo como punto de partida para los conectores reales contra VMware y Proxmox planteados como continuación del trabajo.

Palabras clave: orquestación de máquinas virtuales, infraestructura como código, entornos docentes, herencia en cascada, arquitectura por capas.

DRAFT

Acknowledgments

To my supervisor, Dr. Guillermo Pérez Trabado, for proposing the problem and shaping the design from the beginning, and for trusting me with this project, from which I have learned a great deal.

To my family, for the time they have let me take from them to devote to this work.

To Arturo, who showed me the first real hypervisor in production and planted the seed of curiosity that led me to these studies.

To Anthropic, and to the thousands of people who have devoted the last years to building artificial intelligence. Without this technology it would have been impossible to take on a project of this scope in the time available.

DRAFT

Contents

Abstract	1
Resumen	3
Acknowledgments	5
Contents	7
List of Figures	13
List of Tables	15
1 Introduction	21
1.1 Motivation	22
1.2 Objectives	23
1.3 Adjustments from the original proposal	24
1.4 Report structure	25
2 System concepts	29
2.1 The product through three lenses	29
2.2 Hypervisors, virtual machines and their states	30
2.3 The descriptor: definition plus proxy	32
2.4 The descriptor tree	32
2.5 Cascade inheritance, with examples	33
2.6 Core engine and domain application	34
2.7 Declarative YAML: describing what you want	35
2.8 Reusable resources: define, use, import, closure	37
2.9 Why the same design fits a future RBAC	38
2.10 Summary	39
3 State of the art and technologies	41
3.1 Virtualization, orchestration and the teaching case	41
3.2 Multi-hypervisor abstraction layers	42
3.3 Declarative infrastructure management	43
3.4 Hierarchical RBAC and tenant delegation	44
3.5 Lexical scoping applied to infrastructure	44

3.6	Containers and virtual machines in educational labs	45
3.7	Starting point: the supervisor's previous work	45
3.8	Positioning of UnivOrch	46
3.9	Selected technologies	47
4	Methodology	49
4.1	Process framework: phases and sprints	49
4.2	Reproducible development environment	51
4.3	Quality discipline	52
4.4	Continuous integration and delivery	53
4.5	AI-assisted development	54
4.6	Traceability and knowledge management	55
5	Development, testing and results	57
5.1	Requirements analysis	57
5.1.1	Actors	57
5.1.2	Use cases	58
5.1.3	Functional requirements	58
5.1.4	Non-functional requirements	59
5.1.5	Out of scope for v1	59
5.2	Architectural design	60
5.2.1	Two-layer architecture (DEC-004)	60
5.2.2	C4 context and containers	60
5.2.3	Deployment view	61
5.2.4	Components (as-built)	62
5.2.5	Domain model	62
5.2.6	Patterns applied	64
5.2.7	Declarative load flow (DEC-027)	65
5.2.8	Descriptor state machine (DEC-022, DEC-032)	68
5.2.9	REST as the public boundary	69
5.3	Implementation	70
5.3.1	Orchestrator core	70
5.3.2	Mock connector	71
5.3.3	CLI interface	71

5.3.4	REST daemon	72
5.3.5	Web interface (read-only)	72
5.3.6	Teaching application (layer 2)	73
5.3.7	Packaging and distribution	74
5.4	Testing strategy	74
5.4.1	Unit tests	74
5.4.2	Integration tests	75
5.4.3	Property-based tests	75
5.4.4	REST tests	75
5.4.5	Coverage and metrics	76
5.5	Functional demonstration	76
5.5.1	Starting the daemon	76
5.5.2	Loading the demo tree	77
5.5.3	Inspecting the inheritance	78
5.5.4	Full life cycle	78
5.5.5	Exercising the REST API	79
5.6	Comparison with esobjects and yamlinfr	79
6	Conclusions and future work	83
6.1	Conclusions	83
6.1.1	Fulfilment of objectives	83
6.1.2	Main contributions	84
6.1.3	Known limitations	85
6.2	Future work	86
6.2.1	Next sprints (immediate)	86
6.2.2	Beyond the immediate roadmap	87
6.2.3	Beyond a proof of concept	88
	Appendix A. Administrator's manual	89
A.1	Requirements	89
A.2	Quick installation	90
A.3	Production installation	90
A.4	Real hypervisor configuration (design)	91
A.5	Loading definitions	92

A.6	Day-to-day operation	93
A.7	Monitoring and logs	94
A.8	Backups and recovery	95
A.9	Upgrades	95
A.10	Users and authentication	96
A.11	Troubleshooting	96
A.12	Where to find more information	97
Appendix B. Detailed requirements analysis		99
B.1	Purpose and scope	99
B.2	Actors	99
B.3	Use cases	99
B.3.1	Common	100
B.3.2	Core — Administrator	100
B.3.3	Core — Manager	101
B.3.4	Teaching — Manager (layer 2)	101
B.3.5	Core — End user (student)	101
B.3.6	System — automatic behaviour	102
B.4	Functional requirements	102
B.5	Non-functional requirements	105
B.6	Constraints and assumptions	106
B.7	Out of scope for v1	106
Appendix C. Summary of technical decisions		107
C.1	Summary table	107
C.2	Structural decisions, in detail	108
Appendix D. CLI reference		111
D.1	Invocation modes	111
D.2	Selecting the daemon	111
D.3	Command reference	112
D.3.1	Navigation: <code>cd</code> , <code>pwd</code>	112
D.3.2	Listing: <code>ls</code> , <code>list</code> , <code>tree</code>	113
D.3.3	Inspection: <code>inspect</code> , <code>status</code>	113
D.3.4	Lifecycle: <code>deploy</code> , <code>undeploy</code> , <code>start</code> , <code>stop</code>	113

D.3.5	Loading definitions: <code>load</code>	113
D.3.6	REPL-only commands: <code>connect</code> , <code>help</code> , <code>quit</code>	114
D.4	Examples	114
Appendix E. Conclusiones y líneas futuras (castellano)		117
E.1	Conclusiones	117
E.1.1	Cumplimiento de los objetivos	117
E.1.2	Aportaciones principales	118
E.1.3	Limitaciones conocidas	119
E.2	Trabajo futuro	120
E.2.1	Siguientes sprints (inmediato)	120
E.2.2	Más allá del horizonte inmediato	121
E.2.3	Más allá de una prueba de concepto	122

DRAFT

List of Figures

5.1	Two-layer overview: the daemon <code>univorchd</code> hosts the core; every client (CLI, web GUI, teaching app) speaks REST to it.	61
5.2	C4 level 1 (Context): people and external systems around UnivOrch.	62
5.3	C4 level 2 (Containers): independently runnable units. The daemon <code>univorchd</code> and the CLI <code>univorch</code> are two separate binaries; every client speaks HTTP.	63
5.4	Deployment topology since Sprint 3: clients on tier 1, the daemon container plus its named volume on tier 2, and the hypervisor hosts on tier 3.	64
5.5	C4 level 3 (Components): modules inside <code>univorchd</code> after Sprint 3. Solid green = implemented; dashed grey = designed, not yet implemented.	65
5.6	Domain model and connector contract (C4 code level, view 1/2). Persisted entities (<code>Folder</code> , <code>Descriptor</code> , <code>Job</code>), input documents (<code>DefinitionDocument</code> and friends) and the abstract <code>HypervisorConnector</code>	66
5.7	Engine, service and REST boundary (C4 code level, view 2/2). The <code>OrchestratorAPI</code> Protocol unifies the in-process service and the HTTP client; clients depend on the Protocol, not on either concrete class.	67
5.8	Stages of <code>load / plan</code> : validation rejects <i>before</i> any state change.	68
5.9	Descriptor state machine. Runtime states (running, stopped, paused) belong to the hypervisor and are not persisted by the orchestrator.	81

DRAFT

List of Tables

1.1	Mapping between the eight specific objectives of the project proposal and their status in this proof of concept. None of the items has been dropped from the design; the deferred ones have the necessary extension points in place.	27
3.1	Summary of the implementation technology stack.	48
4.1	The seven phases of the project and their deliverables.	50
4.2	Distribution of automated tests at the close of Sprint 3.	53
5.1	Actors of UnivOrch.	58
5.2	Non-functional requirements of UnivOrch v1 (summary).	59
5.3	Coverage on the core modules (286 tests passing, including the teaching layer).	76
5.4	Comparison of <code>esxobjects</code> , <code>yamlinfr</code> and UnivOrch.	80
6.1	Status of the seven objectives at the close of the proof of concept.	84
6.2	Actors of UnivOrch (detailed).	100
6.3	Non-functional requirements of UnivOrch v1.	105
6.5	Commands provided by the <code>univorch</code> CLI.	112
6.6	Estado de los siete objetivos al cierre de la prueba de concepto.	118

DRAFT

Nomenclature

Acronyms

ABC	Abstract Base Class (Python).
API	Application Programming Interface.
ASGI	Asynchronous Server Gateway Interface.
CI/CD	Continuous Integration / Continuous Delivery.
CLI	Command-Line Interface.
CPU	Central Processing Unit.
CTF	Capture The Flag (cybersecurity competition format).
EI	Escuela de Ingeniería Informática, Universidad de Málaga.
ESXi	VMware's bare-metal hypervisor.
GFS	Grandfather-Father-Son (backup retention policy).
GUI	Graphical User Interface.
HTTP	HyperText Transfer Protocol.
IP	Internet Protocol (address).
IPAM	IP Address Management.
JSON	JavaScript Object Notation.
KVM	Kernel-based Virtual Machine (Linux hypervisor).
LDAP	Lightweight Directory Access Protocol.
RBAC	Role-Based Access Control.

REPL Read-Eval-Print Loop (interactive shell).

REST Representational State Transfer.

SOAP Simple Object Access Protocol.

SSH Secure Shell.

TDD Test-Driven Development.

TFG Trabajo Fin de Grado (final degree project).

UMA Universidad de Málaga.

VM Virtual Machine.

VPN Virtual Private Network.

YAML YAML Ain't Markup Language (data serialization format).

Terms

Lexical closure Rule by which internal references inside a template are resolved from where the template is defined, not from where it is used.

Command Design pattern applied to operations: each action encapsulates `validate()` and `execute()`, enabling dry-run (`plan`), deferred execution and retry.

Connector Adapter between the orchestrator and a concrete hypervisor. Implements the `HypervisorConnector` ABC contract.

deploy / undeploy Create or destroy the VM on the hypervisor from a descriptor (transition between `provisioned` and `deployed`).

Descriptor The orchestrator object representing a VM (its definition plus its state). Analogous to a file descriptor in an operating system.

Folder Grouping node in the tree, container of descriptors and other folders. Supports explicit import of inheritable resources.

Hypervisor (resource) Resource declarable with `define hypervisors::`; names a specific hypervisor instance (with its type and connection data) that descriptors and templates reference with `use hypervisor::`.

Import Mechanism by which a folder states which resources from its parent become visible here; without import, the resource is not accessible from this branch.

Job Persistent record of an operation, with its unique identifier, type, target, status and result. Provides audit and prepares for future asynchronous execution.

load Action that loads a YAML definition document into a destination folder of the tree.

Materialized path Unique identifier of a folder or descriptor; the full path from the root (e.g. `/lab/networks/stude`

Runtime VM state axis from the hypervisor's point of view: `running`, `stopped`, `paused`, `unknown`.

start / stop Power on or off the VM that is already deployed (changes the runtime axis, not the state axis).

State Descriptor state axis from the orchestrator's point of view: `provisioned`, `deployed` (with or without `drifted`), `broken`, `unreachable`.

Template Resource declarable with `define templates::`; offers reusable default values that descriptors reference with `use template::`.

DRAFT

1

Introduction

This work presents **UnivOrch**, a proof of concept for a universal orchestrator of virtual machines aimed at teaching and research environments. The system sits between users and the underlying hypervisors and offers a single way to define, deploy and operate large sets of virtual machines (VMs), regardless of whether they run on VMware, Proxmox or a future platform. Its goal is not to replace the hypervisors but to remove the barrier they impose on non-expert users, in particular professors and students who only need a small subset of what a full virtualization platform exposes.

The work is motivated by the day-to-day situation at the Computer Engineering School (E.T.S. de Ingeniería Informática) of the University of Málaga, where many courses depend on virtual labs and a small team of administrators carries the full operational load. Professors define the VMs that their subject needs and ask the administrator for one set per student; students cannot start or stop their own machines, and any platform change forces the administrator to rewrite the scripts that keep everything running. The TFG starts from two Python libraries written by the supervisor for this exact problem, `esxobjects` and `yamlinfr`, and generalizes them into an orchestrator that is no longer bound to VMware and that exposes a clean public interface for other tools to build upon.

What UnivOrch contributes is best described by what it lets people do, not by any single internal feature. Two outcomes stand out.

First, the institution stops depending on a single hypervisor. The existing solution at the school is tied to VMware; UnivOrch puts every hypervisor behind a common connector contract, so VMware, Proxmox and, in the future, Hyper-V, KVM or a cloud provider can coexist and new ones can be added without touching the core. An institution is no longer captive to one vendor: it can run VMware in the teaching lab and Proxmox in the research lab under the same tool, and migrate without rewriting anything.

Second, people without technical knowledge operate virtual machines on their own. A professor

sets up a whole subject — a folder of machines for every enrolled student — with a couple of commands, and the students start, stop and reset their machines without going through the administrator. One person manages hundreds of VMs without effort that grows in proportion.

These outcomes rest on an architecture designed to scale and to grow. A tree of descriptors with cascade inheritance lets the administrator define the catalogue once and have every node below reuse it, which is what makes hundreds of near-identical machines manageable; the same mechanism is the natural place for a future permission model, and the strict separation between a generic engine and a domain-specific layer means the teaching application could be replaced by a CTF organizer or a research-lab deployer over the same core. None of these ingredients is individually novel — hierarchical organizations exist in OpenNebula and CloudStack, multi-hypervisor abstraction has been provided by libvirt for over a decade, declarative provisioning is the model behind Terraform. The contribution is their combination in a lightweight, single-container tool aimed squarely at delegating VM operation to non-technical users while staying independent of the hypervisor.

This chapter sets the stage. Section 1.1 describes the operational problem the system addresses and the context in which the TFG is carried out. Section 1.2 lists the objectives this proof of concept commits to. Section 1.3 discusses how the final scope differs from the original proposal and why these adjustments preserve the value of the work. Section 1.4 guides the reader through the remaining chapters and appendices.

1.1 Motivation

Practical laboratory work in higher education increasingly relies on virtualization. Reproducing a specific environment for every student of every course is hard to do on physical hardware, while a hypervisor can clone a base machine in seconds and discard it when the practice is over. The technical solution is therefore well understood, but the operational solution is not.

In the current lab at the EI UMA, the administrator is the bottleneck. For every subject, the professor describes the VMs that their students will need, and the administrator manually deploys one identical set of VMs per student. The administrator also assigns IP addresses, keeps track of which student owns which machine, intervenes when a student loses their VM and reverts the environment after the course is over. Each new course adds linearly to this workload. When the underlying platform changes, in our case from a VMware version to the next, the custom scripts that automated parts of the process

need to be rewritten from scratch.

The supervisor of this work, Dr. Guillermo Pérez Trabado, had addressed parts of this problem with two Python libraries developed inside the department:

- `esxobjects` provides a high-level interface to the VMware vSphere SOAP API and hides much of its complexity from the rest of the code.
- `yamlinfr` reads a YAML description of a virtual infrastructure and deploys it through `esxobjects`, with help from an external IP-Address-Management (IPAM) service.

These libraries cover an important part of the problem but stop short in three ways. First, they are tied to VMware: the rest of the school cannot use them on the Proxmox cluster used by the infrastructure-of-systems courses. Second, they offer no abstraction above a single deployment: there is no concept of organizing tens or hundreds of VMs by subject, by group or by student, and no inheritance mechanism to reuse common definitions. Third, the administrator is still the operator: there is no service that professors can use on their own, and certainly no view that students can use to manage their own machines without help.

UnivOrch is built on the experience of these two libraries but re-engineered around a different premise. The orchestrator is a long running service, not a script; it is agnostic of the hypervisor; it maintains its own model of the deployed infrastructure organized as a tree with inheritance; and it exposes a public REST interface that both human users (through a CLI today and a web application in the future) and machines (CI/CD pipelines, GitOps tools, the future teaching-specific layer) can call. The administrator declares the catalogue of hypervisors and templates once; from then on, professors and students operate through their own role-appropriate views, without contacting the administrator for every action.

1.2 Objectives

The objectives of this proof of concept are the following.

01. Hypervisor-agnostic orchestrator core.

Design and implement a service that manages a model of virtual machines (descriptors) and folders, organized in a tree, with no direct dependency on any specific hypervisor.

O2. Declarative model with inheritance.

Define a YAML-based language that allows administrators and professors to describe complete groups of VMs as data, with cascade inheritance of common properties and explicit imports for templates and hypervisor resources.

O3. Connector contract for new hypervisors.

Establish an abstract interface between the core and the hypervisor connectors so that new hypervisors can be plugged in without modifying the orchestrator itself.

O4. REST API as the public boundary.

Expose all orchestration operations through a REST API documented by an OpenAPI specification, suitable for integration with external tools and pipelines.

O5. Command-line client and single-line installation.

Provide a CLI client that consumes the same REST API as any other client, and package the daemon as a Docker image that can be installed with a single command, including its persistent storage.

O6. Engineering quality and discipline.

Build the proof of concept under a strict test-driven development process with high test coverage, static type checking and continuous integration, so the code base is solid enough for the department to keep extending after this work.

O7. Groundwork for the pieces beyond this PoC.

Leave designed (even where not fully implemented) the extension points for the next deliverables: real connectors against the teaching VMware cluster, a basic web interface and the teaching-specific layer-2 application.

A second use of these objectives appears in Chapter 6, where each one is reviewed against what was actually delivered.

1.3 Adjustments from the original proposal

The official project proposal (*anteproyecto*) listed eight specific objectives, sketched as the natural breakdown of a complete orchestrator. The supervisor and the author agreed from the start that

delivering all of them in 296 hours would not be realistic. The proposal itself acknowledged this and described the work as a *proof of concept* whose purpose is to prove the design and leave a foundation for future extensions.

Table 1.1 maps each objective of the proposal to its actual status at the end of this work. The adjustments fall into three broad categories:

- **Delivered as planned.** The orchestrator engine, the declarative model with inheritance, the connector contract, the REST API, the CLI and the packaged distribution are all in place and ready to use.
- **Designed but deferred.** Several pieces have a closed design and the extension points are wired, but the implementation is left for the next iteration. The clearest case is the role-based access control: the model is detailed in the architecture documents and in DEC-011, DEC-021 and DEC-026, but only the *superuser* role is implemented in code. A similar situation applies to the web interface and to the teaching-specific layer-2 application: their design is part of the architecture, but their construction is scheduled for the next sprints.
- **Blocked by external dependencies.** Real hypervisor connectors require access to the VMware cluster of the teaching laboratory. That access was not available during the development period, so the implementation is replaced by a mock connector that behaves identically from the point of view of the orchestrator. The connector contract was validated against the mock; integrating the real VMware connector is the immediate next task once access is granted.

In summary, the proof of concept covers the technical foundation end to end and leaves the remaining pieces with a clear path to follow. The design of the deferred items is not an afterthought: the architecture chapter documents the extension points (the connector abstract base class, the layer-2 boundary, the resolver of inherited permissions, the session and IP-pool repositories) that allow each deferred piece to be added without rewriting the core.

1.4 Report structure

The rest of this report is organized as follows.

Chapter 2 (**System concepts**) introduces the ideas the reader needs in order to follow the rest of the report without gaps. It describes UnivOrch from the point of view of its three kinds of user,

presents the descriptor tree and the cascade inheritance, explains the separation between the engine and the domain application, and walks through the YAML model with worked examples.

Chapter 3 (**State of the art**) reviews the technical context: classic teaching virtual labs, current multi-hypervisor abstractions, declarative configuration systems, hierarchical access control and the specific case of container-based versus VM-based labs. It positions UnivOrch against the closest academic and industrial precedents and identifies the niche it occupies.

Chapter 4 (**Methodology**) describes the working method: the seven-phase plan adapted from a SCRUM-like iterative process, the test-driven development cycle, the supporting tooling (devcontainer, continuous integration, code quality gates) and the use of AI-assisted development as a deliberate and traceable part of the workflow.

Chapter 5 (**Development, testing and results**) is the technical core of the report. It presents the requirements analysis, the architecture of the system in detail and the implementation work organized by layer, followed by the testing strategy and a functional demonstration end to end.

Chapter 6 (**Conclusions and future work**) reviews the objectives against the delivered system, summarizes the contributions and limitations, and outlines the natural extensions of the work.

Five appendices complete the report. Appendix A contains the administrator's manual. Appendix B transcribes the detailed requirements analysis. Appendix C indexes the design decisions referenced throughout the text. Appendix D provides a CLI reference. Finally, Appendix E reproduces the conclusions in Spanish, as required by the regulations of the EI UMA when the report is written in English.

Objective from the proposal	Status	Note
Descriptor model with full life-cycle states	Delivered	Implemented as a two-axis state machine: descriptor state plus runtime state queried on demand.
Linux service with central registry of descriptors	Delivered	REST daemon <code>univorchd</code> packaged as a Docker image.
Hierarchical tree with cascade inheritance and permissions	Delivered (permissions designed)	Tree, inheritance and template closure implemented; RBAC designed but only the superuser role is wired in code.
Hypervisor connectors: mock, VMware, future Proxmox	Partial	Mock connector delivered. VMware and Proxmox connectors are blocked by external access and scheduled as the immediate next task.
Role-based access control with inheritance	Designed, not built	Full design closed (DEC-011, DEC-021, DEC-026). Implementation deferred.
Teaching-specific deployment with IPAM	Deferred	Layer-2 application and IPAM integration documented and left as future work.
REST API and web interface	Partially delivered	REST API delivered and used by the CLI. Web interface (NiceGUI-based) deferred to the next sprint.
Coherence between descriptors and real VMs	Delivered (on demand)	Validation on load, drift detection on demand. A continuous reconciliation loop is documented as future work.

Table 1.1. Mapping between the eight specific objectives of the project proposal and their status in this proof of concept. None of the items has been dropped from the design; the deferred ones have the necessary extension points in place.

DRAFT

2

System concepts

UnivOrch is a service with several moving parts and a vocabulary of its own. Before reviewing the related work and the development of the proof of concept, this chapter introduces the ideas the reader will encounter throughout the rest of the report. The order is deliberately pedagogical: we start from what each user sees, then introduce the underlying domain (hypervisors, VMs and their states), then the orchestrator's central object (the descriptor), then the tree that organizes descriptors, and only after that the mechanisms that hold the tree together (cascade inheritance, declarative loading, lexical closure of reusable resources). The chapter closes with a glance at how the same design fits the role-based access control that v1 leaves as future work.

The aim is that, after reading this chapter, a reader knows what UnivOrch *is and does* without needing to know how it is built. The implementation details are presented in Chapter 5.

2.1 The product through three lenses

UnivOrch is built for three kinds of user, and each of them sees a different slice of the system. Understanding what each one sees, and what each one does *not* see, is the easiest way to grasp what the system is for.

The end user (a student).

The end user opens the web interface, logs in, and sees a curated list of the virtual machines assigned to them. The list is small: typically the machines for one or two courses. The user can start, stop and (when offered) reset their own machines. They do not see the catalogue of base templates, the underlying hypervisors, the hierarchy of folders or the orchestrator's vocabulary; what they see is

a domain-specific view designed for them, in our case the metaphor of a virtual *desk* that contains *computers* (DEC-009). The whole point of UnivOrch from this perspective is that the student operates without depending on the system administrator.

The manager (a teacher).

The manager (in the teaching application, a professor) sees a larger slice. They see the branch of the system that belongs to their courses: the folders for their subjects, the descriptors for each student, the templates they have made available to those students. They can create folders and descriptors inside their branch, change properties, deploy or undeploy whole subjects in a single operation, and audit the history. They do not see the catalogue of users, the global system configuration, the credentials of hypervisors or the branches that belong to other teachers.

The administrator.

The administrator sees everything. They define the hypervisors and the base templates that everyone else uses, they manage the user catalogue, they grant or revoke access to other roles, and they respond to incidents (for example, recovering a descriptor that was left inconsistent after a failed deployment). The administrator is also the person who installs and operates the daemon itself.

A single person can, of course, hold different roles in different parts of the system; the role lives on the folder, not on the user record. We make this precise in Section 2.9 at the end of the chapter. For now, it is enough to think of three different views of the same product.

2.2 Hypervisors, virtual machines and their states

A *hypervisor* is a piece of software that lets multiple virtual machines share the same physical hardware. VMware ESXi and Proxmox VE are two examples that this work targets. Each VM believes it owns its CPU, memory, disks and network adapters; the hypervisor presents that illusion and arbitrates access to the real hardware. A hypervisor lives on a single host (or a small cluster of hosts) and exposes an API that lets external programs create, destroy, start and stop VMs.

A *virtual machine*, from the hypervisor's perspective, has three runtime states relevant to this work:

- **running**: the VM is powered on and executing.

- **stopped**: the VM exists in the hypervisor but is powered off.
- **paused**: the VM is suspended in place; its memory is preserved and a resume operation can return it to **running**.

These three states are well known and shared by every hypervisor. UnivOrch reports them but does not invent them.

The new axis: provisioned versus deployed.

The most important concept of this chapter is that UnivOrch introduces a *second state axis* that does not exist at the hypervisor level. This second axis is the orchestrator's own view of the relationship between the descriptor and the real VM, and it has four values (DEC-022, DEC-032):

provisioned – the orchestrator has the descriptor (definition and identifier) but the *hypervisor has no VM yet*. The VM has been described but not built. This is the natural initial state of every descriptor.

deployed – the descriptor has a matching VM in the hypervisor. This is the state from which the three runtime states above become meaningful: a deployed VM may be running, stopped or paused.

broken – an operation (typically a deploy or undeploy) failed in a way that left the orchestrator and the hypervisor out of sync. Recovery requires an explicit *force-undeploy*.

unreachable – the hypervisor cannot be reached; the actual state of the VM is therefore unknown. The orchestrator marks this state reactively, when a request fails because of a connectivity error.

This second axis is the conceptual key of UnivOrch. It lets a professor describe an entire subject *before* any VM is built, keep the descriptions versioned and reproducible, and then either deploy them all at once or let each student deploy their own machine on demand. Without the **provisioned** state, the only way to describe an environment would be to instantiate it, which is what forces every traditional setup script to be tied to the moment of deployment.

A descriptor in **provisioned** consumes no hypervisor resources. A descriptor in **deployed** corresponds to a VM that exists, takes disk space and may consume memory and CPU. Moving from **provisioned** to **deployed** and back is, from the user's point of view, the central operation of the system.

Fig. 5.9 on page 81 of Chapter 5 shows the formal state machine that ties both axes together.

2.3 The descriptor: definition plus proxy

The main object UnivOrch manipulates is the *descriptor*. The name is borrowed from operating systems: in a Unix-like kernel, a file descriptor is a small integer that points to an open file without being the file itself; the application reads and writes through the descriptor, the kernel takes care of the actual file. The analogy is direct (DEC-005). In UnivOrch:

- The descriptor is the orchestrator's handle to a virtual machine.
- The descriptor is *not* the virtual machine. The VM lives in the hypervisor.
- Users interact with the descriptor; the orchestrator takes care of talking to the hypervisor on their behalf.

A descriptor is two things at once. On the one hand, it carries the *definition* of the VM it represents: the base template it should be cloned from, the hardware parameters (CPU, memory, disks), the target hypervisor and any property the system needs in order to build the VM. On the other hand, it serves as a *proxy* to the deployed VM: when the descriptor is **deployed**, it records the identifier of the VM in the hypervisor, so the orchestrator knows where to send operations like **start** or **stop**.

The distinction between definition and proxy is what makes the **provisioned** state meaningful. A provisioned descriptor carries the full definition but no proxy; a deployed descriptor carries both. Moving between the two states is what a **deploy** or an **undeploy** does.

A descriptor lives at a single path inside the descriptor tree, and that path is its identifier. We turn to the tree next.

2.4 The descriptor tree

UnivOrch organizes descriptors in a tree. Each node of the tree is either a *folder* (an internal node) or a *descriptor* (a leaf). The structure is identical to a Unix file system, and the identifiers follow the same convention: a forward slash separates levels, the root is `/`, and a node is identified by the concatenation of names from the root to the node.

For example, the system that the proof of concept ships as a demo contains:

<code>/lab/</code>	<code>folder, administrator's workspace</code>
--------------------	--

<code>/lab/networks/</code>	folder, one subject
<code>/lab/networks/student01</code>	descriptor, a single student's VM
<code>/lab/networks/student02</code>	descriptor, another student's VM
<code>/lab/networks/student03</code>	descriptor, another student's VM

The tree is the conceptual scaffolding of the whole system. It gives identity to every node (the path), gives structure to access control (you have permissions on a branch), gives a natural unit for batch operations (*deploy the whole subject*), and – as we see in the next section – gives a place for shared configuration.

The tree also gives a natural vocabulary for the teaching application. A subject becomes a folder. A student becomes either a descriptor (when each student has a single VM) or a folder underneath the subject (when each student has several VMs). The same tree could be used for a CTF competition or a set of laboratory experiments by changing only the labels in the domain application.

Folders can hold their own properties (description, the resources they make available to their descendants, the IP pool they assign addresses from) and zero or more descriptors. A descriptor cannot contain other descriptors; descriptors are the leaves of the tree.

2.5 Cascade inheritance, with examples

The reason the tree is useful is that *properties cascade*. Anything defined at a folder is inherited by its descendants. The mechanism is the closest the system has to a magic ingredient, so we present it in two passes: first the rules, then two worked examples.

The three rules.

The merge rule between parent and child depends on the type of the field (DEC-026):

- **Scalar fields** (an integer, a string): the child's value, if it exists, overrides the parent's.
- **List fields**: the child's list *accumulates* on top of the parent's. An item that appears in the parent's list is still there for the child unless the parent no longer offers it.
- **Dictionary fields**: the parent's and child's dictionaries are merged *recursively*, applying the same three rules to each key.

There are well-known exceptions where the default does not fit. The IP pool is one: a pool is a coherent block (network, mask, gateway, range) and merging it field by field would yield nonsense. For those cases the engine accepts an explicit override that replaces the whole field rather than merging it.

Example 1: a simple override.

Suppose the administrator declares, at the folder `/lab`, a default of `cpu: 2` for every machine that lives below. The folder `/lab/networks` adds nothing. The descriptor `/lab/networks/student01` does not state a CPU value, so it inherits `cpu: 2`. The descriptor `/lab/networks/student03` explicitly declares `cpu: 4`. Both inherit through `/lab/networks`; `student01` ends up with two CPUs, `student03` with four.

Example 2: an accumulated list.

The folder `/lab/networks` declares an `import: list` containing `[linux-vm]`, meaning “the template `linux-vm` is available to my descendants”. The folder `/lab/networks/group-A` adds `import: [windows-vm]`. A descriptor inside `group-A` sees both `linux-vm` and `windows-vm`, because the imports of the two folders have accumulated as the list rule prescribes.

Implications.

Cascade inheritance is what makes the system practical at scale. The administrator declares the catalogue of base templates and the hypervisor configuration *once*, in a high-level folder, and every descriptor below inherits them. A subject of a hundred students is then a hundred near-identical descriptors that share the bulk of their configuration. The teacher edits the shared configuration in one place; the change reaches every student in the next deployment.

2.6 Core engine and domain application

UnivOrch is built in two layers (DEC-004). The lower layer is the *generic engine*: it knows about folders, descriptors, jobs, inheritance and hypervisors, but it knows nothing about subjects, students or any specific domain. The upper layer is the *domain application*: it gives meaning to the generic tree in

terms of one concrete use case.

The proof of concept ships a single domain application – the teaching one – but the engine is general. The same engine could serve a CTF (where the tree models teams instead of subjects), a set of conference workshops (where each attendee gets a temporary VM from a shared template), or a research laboratory (where each experiment has its own folder with its own machines and is recycled when the experiment ends). The domain application is, in all of these cases, a thin client that translates the vocabulary of the domain (*subject*, *student*, *team*, *experiment*) into the generic operations of the engine (create folder, create descriptor, deploy, undeploy).

The boundary between the engine and the domain application is the public REST API of the daemon. The teaching application is, from the engine's point of view, an HTTP client like any other; the engine has no special path for it.

The separation matters for two reasons. First, the engine can be reused without modification by a different domain application written by a third party. Second, the engine is much easier to maintain when it stays free of teaching vocabulary; the specialization happens once, at the boundary, and never leaks inwards.

2.7 Declarative YAML: describing what you want

UnivOrch follows a declarative model. The user does not say “create this folder, then create that descriptor, then assign this template” one step at a time. The user describes the desired state in a YAML file, and the system computes the operations needed to bring the tree into that state. The model is the same one used by Terraform and Ansible (Chapter 3); the operation that applies it in UnivOrch is called `load`.

The load operation.

`load` takes a YAML file and a destination path. It reads the YAML, validates it against the current tree, and creates the new folders and descriptors at the destination. The destination is any folder in the tree, not only the root; this is what makes a YAML file *portable*.

For example, the demo file `demo/setup.yml` describes a folder `lab` that contains a folder `networks` that contains three student descriptors. The administrator runs:

```
univorch load demo/setup.yml /
```

and the structure appears under the root. A different administrator, on a different installation, can run:

```
univorch load demo/setup.yml /research/site-B/
```

and the same structure appears under that other folder. The paths inside the YAML are relative to the destination; the file is a reusable template.

What a definition looks like.

The YAML file is a nested dictionary that follows the structure of the tree. The demonstration file the proof of concept ships looks like this:

```
kind: definition
version: "1"
folders:
  lab:
    description: "Computer Science Lab"
    define hypervisors:
      mock01: { type: mock }
    define templates:
      linux-vm:
        use hypervisor: mock01
        base_vm: linux-base
    folders:
      networks:
        description: "Computer Networks course"
        import: [linux-vm]
        descriptors:
          student01: { use template: linux-vm, cpu: 2 }
          student02: { use template: linux-vm, cpu: 2 }
          student03: { use template: linux-vm, cpu: 4 }
```

The structure is straightforward. Folders nest under **folders**; descriptors live under **descriptors**. Resources that are made available to descendants (hypervisors and templates) are declared with

define. Resources that the folder *borrow*s from its parent are declared with **import**. Local properties of a descriptor (here, the `cpu` of each student) live next to the references.

The vocabulary of **define**, **use** and **import** deserves its own section.

2.8 Reusable resources: **define**, **use**, **import**, **closure**

The descriptor tree is more than a hierarchy of names; it is also a hierarchy of *scopes* in which resources are declared, used and imported. Three keywords govern the model:

define X: declares a resource at this folder. The resource exists from now on inside this folder and below. The two resources supported in v1 are **hypervisors** and **templates**. Future extensions will add **datastores** and IP **pools**, with the same grammar.

use X: references an existing resource by name. A descriptor that says **use template: linux-vm** inherits the fields of the template; a template that says **use hypervisor: mock01** targets that hypervisor. The name is resolved in the scope where the resource was defined, not where it is used (we return to this below).

import: is the visibility control. A folder that imports a name from its parent makes that name visible to its descendants. A name that is not imported is invisible below, even if the parent defined it. This lets an administrator publish a large catalogue of templates while keeping each course's branch limited to the templates the teacher chose to use.

The interaction between **define**, **use** and **import** mirrors the interaction between **define**, **use** and **import** keywords in many programming languages, which is not a coincidence. The next idea makes the analogy explicit.

Lexical closure of templates.

When a template defined in folder *A* is imported into folder *B*, and a descriptor inside *B* uses the template, every reference inside the template (for example **use hypervisor: mock01**) is resolved *starting from A*, not from *B*. The template carries with it the scope where it was defined; we say it *closes over* that scope. This is the same behaviour a closure has in a programming language: the function definition captures the environment in which it was written.

The motivation is purely practical. A teacher who imports the template `linux-vm` from the administrator's catalogue is not expected to also import the underlying hypervisor; they should just have to write `use template: linux-vm` and have it work. The hypervisor reference inside the template is the administrator's business, not the teacher's. Lexical closure makes this just work.

Example.

The demonstration file above declares `mock01` (the hypervisor) and `linux-vm` (the template) at the folder `/lab`. The folder `/lab/networks` imports only `linux-vm`; it does not need to import `mock01`. When `student01` says `use template: linux-vm`, the engine merges the template's fields into the descriptor; the template's own `use hypervisor: mock01` is then resolved against `/lab`, where `mock01` lives, even though the descriptor itself lives in `/lab/networks`.

2.9 Why the same design fits a future RBAC

The cascade inheritance described above is not specific to configuration fields. It applies, with the same rules, to *role assignments*. UnivOrch defines three roles (DEC-011):

superuser – the system administrator. Assigned at the root; full visibility and control.

manager – a teacher or equivalent. Assigned at the folder of a subject; can manage the descendants of that folder.

end_user – a student or equivalent. Assigned at the folder of a workstation or set of workstations; can operate the descriptors below.

A role assignment is attached to a folder, not to a user record (DEC-021). The user record itself only carries identity information (name, password, e-mail). Whether the user is a teacher in one branch and a student in another is decided by which folders mention them and in which role.

Two consequences follow directly from cascade inheritance. First, an assignment made at a high level reaches every descendant automatically: declaring a user as **end_user** at a folder gives them access to every workstation inside it. Second, a more specific assignment lower down can override the inherited one if needed: a user who is an **end_user** of an entire subject can additionally be made a **manager** of a single sub-folder of that subject.

The proof of concept does not implement the full RBAC in code; only the `superuser` role is realistic in v1, and the service assumes it runs in a trusted network. The point of this section is that the model is consistent with the rest of the design and that *the engine does not need to grow new mechanisms to support roles*. The same cascade-inheritance code that resolves configuration fields will resolve role assignments. When the implementation arrives, it slots into the existing engine without rearchitecting it.

2.10 Summary

The reader who reaches this point has the conceptual map of UnivOrch:

- Three kinds of user (student, teacher, administrator), each with a different view of the system.
- A second state axis on top of the hypervisor's runtime states: `provisioned`, `deployed`, `broken`, `unreachable`.
- The descriptor as the orchestrator's handle, carrying both the definition of a VM and the proxy to the deployed one.
- The descriptor tree as the scaffolding that gives identity, structure and inheritance to every node.
- Cascade inheritance with three type-driven rules and a small set of well-defined exceptions.
- Two layers, a generic engine and a domain application, with the REST API as the boundary between them.
- A declarative YAML model and a `load` operation that accepts any destination in the tree.
- Three keywords – `define`, `use`, `import` – that turn the tree into a hierarchy of scopes, plus a lexical closure that lets templates travel with their definition environment.
- A role-based access control whose mechanics fit the same cascade and are designed for a future implementation.

The following chapters take this conceptual map for granted. Chapter 3 compares UnivOrch with the closest existing systems and identifies where the model is original. Chapter 4 describes how the proof of concept was built. Chapter 5 is the technical core of the report: it documents the architecture and the implementation in detail. Chapter 6 closes the report against the objectives.

DRAFT

3

State of the art and technologies

This chapter places UnivOrch in the landscape of existing systems and ideas. We approach it from several angles: the closest academic precedent of virtual labs for higher education, the lineage of multi-hypervisor abstractions, the broader move from imperative to declarative infrastructure management, the well-known field of role-based access control with hierarchy, and a recurring discussion that surrounds any educational lab today, namely whether containers or VMs are the right substrate. We then describe the two libraries that the supervisor had developed before this work and that serve as the operational starting point. We close with an honest positioning of UnivOrch against everything that exists, and with the concrete technologies adopted for the implementation.

The aim throughout is to be precise about what is already available and what is not. Several pieces of UnivOrch borrow from prior art and the report acknowledges those debts; the contribution lives in the specific combination, not in any individual feature.

3.1 Virtualization, orchestration and the teaching case

The terms *hypervisor*, *hypervisor manager* and *orchestration* are often used loosely. In this work we use them in a strict sense. A *hypervisor* (VMware ESXi, Proxmox VE, KVM, Hyper-V) creates and runs virtual machines on a single host. A *hypervisor manager* (VMware vCenter, Proxmox cluster) groups several hypervisors of the same family under a single administrative view, but still speaks the vocabulary of the hypervisor. An *orchestrator* sits one level above: it manages the life cycle of

VMs across one or more managers, exposes its own model of the resources, and can speak to several heterogeneous hypervisors through adapters.

The orchestrator can be *imperative* (the user issues `create`, `start`, `stop` commands) or *declarative* (the user describes the desired state and the orchestrator computes the operations needed to reach it). Burgess established the theoretical framing of this latter model in his *theory of system administration* [3], introducing the notions of *convergence* between desired and actual state and *configuration drift*, which form the conceptual basis of Terraform, Ansible and Kubernetes alike. UnivOrch adopts the declarative model with on-demand application; continuous reconciliation is mentioned in the closing chapter as a natural extension.

The case that justifies a dedicated orchestrator is well documented in the academic literature on virtual labs: systems that let professors define VM environments and let students access their own machines on demand have been built and operated at scale, from early remote labs such as Tele-Lab [16] to cyber ranges like KYPO [15]. What these systems share with UnivOrch is the goal — delegated, reproducible VM environments for education — and what UnivOrch adds is a different engineering stance: a hypervisor-agnostic engine instead of a stack tied to one platform, a declarative YAML model with cascade inheritance instead of an imperative reservation system, and a strict separation of the domain layer (teaching, CTF, research) from the engine through a clean public boundary.

3.2 Multi-hypervisor abstraction layers

Abstracting several hypervisor families behind a common interface is not a new idea. The reference is `libvirt` [2], an open-source library that exposes VMware, Hyper-V, KVM and Xen behind a single API. Its model inspired the abstract `HypervisorConnector` interface used in UnivOrch (Chapter 5), and the connector contract follows the same general shape: a small set of life-cycle operations (`clone`, `delete`, `start`, `stop`, `get_status`) plus a way to expose hypervisor-specific parameters.

At a higher level, several open-source orchestrators already provide multi-hypervisor coverage. OpenNebula [8] supports KVM, LXC, vSphere and Firecracker and organizes users in two-level groups (groups and Virtual Data Centers); Apache CloudStack [14] supports KVM, vSphere, XenServer and Hyper-V and organizes both the infrastructure (Region > Zone > Pod > Cluster) and the user space (Domain > Account) in trees; oVirt [9] focuses on KVM but exposes Data Center > Cluster > Host as a hierarchy with inheritance of machine type and CPU model from the cluster. The Distributed

Management Task Force, on the standards side, offers Open Virtualization Format (OVF) [5] for cross-hypervisor VM portability.

The conclusion is that multi-hypervisor abstraction *per se* is not a novel contribution; what UnivOrch contributes in this area is a much lighter integration model. OpenNebula and CloudStack are substantial systems that assume a dedicated operations team. UnivOrch keeps the same connector pattern but ships as a single container intended to be installed in one command and operated by a single person.

3.3 Declarative infrastructure management

The declarative model has become the standard in the infrastructure-as-code field. Rahman, Mahdavi-Hezaveh and Williams present a systematic mapping study [10] that identifies the main research themes in this area, and their companion work on defective IaC scripts gives empirical grounding to the discussion of correctness. The dominant industrial tool, Terraform, organizes infrastructure in *modules* that can be composed through inputs and outputs, but variables do not inherit across module boundaries automatically; inheritance has to be wired by hand.

The closest current system to what UnivOrch tries to achieve is Crossplane [4], which runs inside a Kubernetes cluster and uses Composite Resource Definitions (XRDs) and Compositions to abstract any cloud or platform resource behind a simplified Claim. Crossplane resolves elegantly the separation between a generic engine (Kubernetes plus the Crossplane controllers) and a domain layer (the XRDs that each organization defines for its own platform). This work cannot ignore Crossplane: the architectural pattern is similar in spirit. UnivOrch differs in three points: it does not require a Kubernetes cluster, which is a practical advantage in environments with one to a few hosts; it uses a tree-structured domain model with cascade inheritance instead of composition graphs; and it accepts that it is much less mature and less extensible than Crossplane.

In the specific niche of *GitOps over VMs*, Red Hat has documented an integration of KubeVirt with OpenShift GitOps [12] that treats VM definitions as code in a Git repository. This is a useful reference for the future-work section of this report; UnivOrch does not implement Git triggers in v1 but its declarative YAML interface and the public REST API leave the door open to the same workflow.

3.4 Hierarchical RBAC and tenant delegation

Role-Based Access Control with role hierarchy is a classical topic, formalized by Sandhu et al. in [13] and later standardized by NIST and ANSI. The RBAC1 model directly maps to what UnivOrch implements at the design level: a hierarchy of roles in which the assignment at a higher level cascades to descendants, with the possibility of overriding at any node.

Delegation of management to non-administrator users with a simplified view is also common in industry products. OpenNebula exposes a *Cloud View* for end users in its Sunstone interface; VMware Cloud Director offers Tenant Portals; Nutanix Prism incorporates Self-Service blueprints. In all these systems, an ordinary user receives a curated subset of operations corresponding to their role, without exposure to the underlying machinery. UnivOrch embraces this same principle and goes further in one specific dimension: the metaphor presented to the student is intentionally domain-specific (a *desk* that contains *computers*) and hides not only the technical machinery but also the orchestrator's own concept of a tree. This is a contribution at the level of user-experience design, not at the level of access-control mechanism; we present it as such in Chapter 5.

The closest pattern in the open-source space is the Hierarchical Namespace Controller (HNC) of Kubernetes [7], which introduces a tree of namespaces with automatic propagation of RBAC roles, resource quotas and network policies. HNC is also a good reference for the inheritance semantics chosen in UnivOrch.

3.5 Lexical scoping applied to infrastructure

A small detail of UnivOrch's inheritance model connects to programming-language theory and is worth a brief note. When a template is imported from a parent folder into a child folder, references inside the template (e.g. the hypervisor it targets) are resolved *against the folder where the template was defined*, not against the folder where it is used — the behaviour of a *lexical closure*, where the environment in which a function was defined accompanies the function wherever it travels.

Lexical scoping is a classic idea from programming languages, treated pedagogically in SICP [1] and alive today in configuration languages such as Jsonnet, CUE and Dhall. Applying it to a concrete model of infrastructure resources is a minor but, as far as we found, uncommon touch; it is a detail of the inheritance machinery (Section 3.8), not a headline contribution.

3.6 Containers and virtual machines in educational labs

A reasonable reader may ask, in 2026, why a TFG on VM orchestration makes sense at all when container-based stacks are dominant. The answer comes from the specific requirements of practical work in computer science education. Several recent works are explicit on the matter. The KYPO cyber range [15] argues that hands-on training in network security and operating-system internals demands access to a privileged kernel and to arbitrary networking, neither of which a non-privileged container can offer: containers suffice for many introductory scenarios, but full VMs remain necessary for kernel-level and complex-networking exercises. The Tele-Lab project [16] of HPI Potsdam is an earlier example of a VM-based learning environment with a long operational history.

The Labtainers framework [6], in contrast, demonstrates that the container approach is perfectly viable when the lab focuses on application-level security and does not require kernel modifications, and it offers an interesting model of parameterized, per-student lab instances.

The conclusion is that the choice of substrate depends on the nature of the lab. UnivOrch focuses on the VM case for two reasons: the existing labs at the EI UMA rely on VMware ESXi and Proxmox; and the kinds of exercises that drive the most operational pain at the school (computer networks, operating systems, infrastructure administration) are precisely the ones that require full VMs.

3.7 Starting point: the supervisor's previous work

The TFG starts from two Python libraries developed by the supervisor, Dr. Guillermo Pérez Trabado, inside the Department of Computer Architecture of the University of Málaga. They are unpublished internal software but solve a real operational need at the school.

`esxobjects` is a Python library that wraps the VMware vSphere SOAP API behind a friendlier object-oriented interface. It takes care of authentication, session management and the most common operations on VMs and templates. It is tied to VMware by construction.

`yamlinfr` is a higher-level layer built on top of `esxobjects`. It reads a YAML description of an infrastructure of VMs and deploys it onto VMware. It also interacts with an external IPAM service to allocate IP addresses. It is the closest operational antecedent of UnivOrch: it already adopts a YAML-based declarative model and the idea of deploying a whole infrastructure at once.

UnivOrch generalizes this experience along three axes. First, the hypervisor is no longer fixed: a

connector abstract class allows VMware, Proxmox and a development mock to coexist behind the same interface. Second, the YAML model is enriched with a tree of folders, cascade inheritance and template closures, which the previous libraries did not provide. Third, the system becomes a long-running service with a public REST API, accessible to multiple clients (CLI, future web GUI, external tools), rather than a one-shot deployment script. A qualitative comparison between `esxobjects`, `yamlinfr` and UnivOrch closes Chapter 5.

3.8 Positioning of UnivOrch

Summarizing the sections above, the original contribution of UnivOrch is not the appearance of any individual feature in isolation. Hierarchical organizations of resources and users are provided by OpenNebula, CloudStack, vCloud Director, oVirt and the Kubernetes HNC; declarative provisioning is the standard model behind Terraform and the various flavours of GitOps; multi-hypervisor adapters have been provided by libvirt for over a decade; lexical scoping in configuration languages already exists in Jsonnet, CUE and Dhall; tenant delegation with simplified user views is offered by every major commercial cloud platform.

What UnivOrch contributes is the synthesis of these ingredients in a lightweight, single-container tool explicitly aimed at delegating VM operation to non-technical users while staying independent of the hypervisor. Concretely, the defensible points, in order of value:

1. **Hypervisor independence with a low barrier to extension.** Multi-hypervisor support is not novel in itself (libvirt, OpenNebula, CloudStack all provide it), but UnivOrch delivers it in a tool light enough to be installed and operated by one person, and its connector contract makes adding a new hypervisor type a matter of writing one class. For an institution this is the difference between being locked to one vendor and being able to mix and migrate freely. It is also the concrete generalization of the supervisor's VMware-only starting point (Section 3.7).
2. **Delegation to non-technical users.** Simplified tenant views exist in every major cloud platform, but UnivOrch is built around this from the ground up and pushes it further: the end user (a student) operates VMs through a domain-specific metaphor (*desks* and *computers*) that hides not only the hypervisor but the orchestrator's own concepts, and a manager (a professor) runs a whole subject without touching the administrator. This is as much a user-experience contribution as an engineering one, and we present it as such.

3. **A domain-agnostic engine with a thin domain layer.** The strict separation between a generic core and a domain-specific application (teaching, and tomorrow CTF or research labs) is implemented as a thin client of a public REST API rather than as a controller on top of Kubernetes. The pattern is familiar from Crossplane; the contribution is to show that an equivalent decoupling is achievable in a much lighter deployment.
4. **An inheritance model that scales the configuration.** A tree with cascade inheritance by data type lets the catalogue be defined once and reused across hundreds of nodes, and is the natural seat of a future permission model. One detail of this model is less common in the surveyed systems: templates resolve their internal references from where they are defined, not from where they are used — a lexical-closure behaviour borrowed from programming languages (Section 3.5). It is an elegant piece of the machinery rather than the headline.

In terms of maturity and engineering depth, the most demanding point of comparison among the systems surveyed is Crossplane: it solves the engine/domain separation elegantly, but on top of a full Kubernetes cluster. UnivOrch is far from that maturity; its claim is the opposite trade-off — the same kind of decoupling, plus hypervisor independence and delegation to non-technical users, in a tool light enough for one person to install and run.

3.9 Selected technologies

Table 3.1 summarizes the technologies adopted for the implementation. The choices are documented in detail in `docs/technologies.md` of the project repository and in the design-decision DEC-033; we restate the rationale here.

Two design alternatives are worth mentioning briefly. SQLite was considered for persistence; it was rejected in favour of TinyDB because the descriptor data is naturally document-shaped (deeply nested YAML fragments), not relational, and TinyDB removes the need for an ORM in the proof of concept. Click was considered as a CLI framework; cmd2 was preferred because it offers an interactive REPL in addition to one-shot commands without extra code, which matches the experience the end user is expected to have.

The implementation is presented in detail in Chapter 5.

Layer	Selected technology	Why
Language	Python 3.12	Same language as the supervisor's libraries; native abstract classes, type hints and <code>@override</code> ; well supported in a long-lived container.
Web framework	FastAPI + Uvicorn [11]	Generates an OpenAPI specification automatically; strict typing integrates with Pydantic; ASGI test client is standard.
Domain modelling	Pydantic v2	Declarative model, validation at the boundary, JSON serialization for free, used by FastAPI internally.
HTTP client (CLI)	httpx	Same API as <code>requests</code> , with async support and a strict test transport; replaceable through dependency injection in tests.
Persistence	TinyDB (PoC)	A single JSON file as the database keeps the proof of concept self-contained. A MongoDB-backed repository is the natural successor for production.
CLI	cmd2	Bash-style and REPL modes from the same command set; argparse integration; well-tested behaviour for tab completion and help.
Quality	Ruff, mypy-strict, pytest, Hypothesis	Fast linter and formatter; strict static typing; full unit and integration test coverage; property-based testing on the pure Resolver.
Tooling	uv + Docker	<code>uv</code> for reproducible Python environments; multi-stage Docker for a minimal runtime image; image hosted in GitHub Container Registry.

Table 3.1. Summary of the implementation technology stack.

4

Methodology

This chapter describes the working method behind the proof of concept. It is not a generic overview of agile or test-driven development: the goal is to record the specific choices that were made and the reasons for them, so that any reader can reproduce the process or take it as a basis for future iterations.

The method rests on four pillars. First, a phased process with a formal deliverable per phase, complemented by short iterations (sprints) inside the implementation phase. Second, a single reproducible development environment that works identically on a local machine, on a remote server and on cloud workspaces. Third, a strict code-quality discipline supported by continuous integration. Fourth, an explicit and traceable use of AI-assisted development.

The four pillars are tied together by a set of versioned context files that act as the operational memory of the project: the chronological journal, the technical-decisions log, the pending-ideas backlog and the development guidelines. These files live in the `claude/` directory of the repository and are loaded by the AI assistant at the start of every session, but they also serve as the only written record of the rationale behind every design and code change.

4.1 Process framework: phases and sprints

The work is organized in seven phases, each with a specific deliverable. Table 4.1 lists them. The structure follows the classical engineering breakdown (problem definition, requirements, architecture, technology selection, environment, implementation, evaluation) and assigns each phase to a stable artefact in the repository. The first five phases are sequential because their outputs feed the next; the sixth phase contains internal iterations, and the seventh runs in parallel with the sixth from the moment the documentation work begins.

Phase	Activity	Deliverable	Status
1	Problem definition	<code>docs/vision.md</code>	Completed
2	Requirements analysis	<code>docs/requirements.md</code>	Completed
3	Architecture design	<code>docs/architecture.md</code> <code>docs/diagrams.md</code>	+ Completed
4	Technology selection	<code>docs/technologies.md</code>	Completed
5	Environment configuration	Working devcontainer <code>docs/environment.md</code>	+ Completed
6	Iterative development (TDD)	Working software (sprints 1–3)	In progress
7	Evaluation and report	This document	In progress

Table 4.1. *The seven phases of the project and their deliverables.*

Inside Phase 6, the work is split into *sprints* of two to three weeks each, with a clear scope agreed at the start and a formal close in the journal. Three sprints were completed during the development period:

- **Sprint 1** (Minimum viable core, CLI). Delivered the descriptor tree, the cascade-inheritance model, the connector contract, the mock connector, the YAML loader, the Job engine and a working CLI. Closed with a runnable demo for the supervisor.
- **Sprint 2** (Resolver and template closure). Delivered the lexical-closure resolver, the connector type registry and the live-session pool. Closed with the demo tree deployable end-to-end without any inline hypervisor information in the descriptors.
- **Sprint 3** (REST daemon and packaged distribution). Delivered the FastAPI daemon, the pure-HTTP CLI client, the installable Docker image hosted in the GitHub Container Registry, a single-line installer script and a tutorial for the supervisor.

The journal (`claude/diario.md`) records every decision in chronological order; the technical-decisions log (`claude/decisiones.md`) keeps the numbered DEC entries; both are cross-referenced. The proof of concept reaches the end of Sprint 3 with 244 automated tests passing, 2,522 lines of source code under `src/univorch/` and 2,796 lines of tests under `tests/`. Chapter 5 describes the delivered software in detail.

4.2 Reproducible development environment

The development environment is defined by a single `.devcontainer/devcontainer.json` file in the repository. The same file produces an identical environment in three setups:

- Visual Studio Code with local Docker (the standard option).
- Visual Studio Code with the *Remote – SSH* extension connected to a Linux server, with Docker running on the server. This is the recommended option for Phase 6 because the server has direct network access to the teaching VMware and Proxmox hypervisors and lifts the hour limits of cloud workspaces.
- GitHub Codespaces in the browser. Useful for short sessions or for working from a machine without Docker.

The base image of the devcontainer is the official Microsoft Python 3.12 image. The container installs `uv` as the package manager and synchronizes all development dependencies declared in `pyproject.toml` with the `dev` extra. Editor configuration, linter settings and useful VSCode extensions are also part of the file. A developer who clones the repository and opens it in VSCode is ready to work within minutes, without any manual installation.

The development image and the production image are deliberately different. The development image is convenience-oriented (full toolchain, debugger, Claude Code extension). The production image is a multi-stage minimal build (Python 3.12 slim, the application code and `uv` from a layer cache) of around 200 megabytes. The production image is published in the GitHub Container Registry (`ghcr.io/clamaveruma/univorch`) on every tagged release.

The repository follows a Python source layout with a single distribution package, `univorch`, and the test suite split in unit and integration tests:

```
univorch/
  src/univorch/      application code
  tests/unit/        isolated unit tests
  tests/integration/ tests with TinyDB + MockConnector
  demo/             YAML examples and demo guide
  docs/             phase deliverables (English)
  claude/           project memory and AI context (Spanish)
```

<code>.devcontainer/</code>	reproducible dev environment
<code>.github/workflows/</code>	CI/CD pipelines
<code>Dockerfile</code>	production image
<code>docker-compose.yml</code>	service composition
<code>pyproject.toml</code>	unified project configuration

4.3 Quality discipline

The codebase was developed under a strict test-driven discipline. Every public function has at least one test, every Command is tested with both an accepting and a rejecting case, and every Repository is tested against an in-memory TinyDB. The pure Resolver, being a function over plain data, is also exercised by property-based tests written with Hypothesis, which automatically generates trees of folders and descriptors and checks invariants such as “a child folder cannot shadow a hypervisor that was never imported.”

The four quality gates run on every commit. They are also enforced in continuous integration; a failed check blocks the build.

Ruff – linter and formatter in a single tool. Replaces `flake8`, `isort` and `black`. Configured in strict mode for the application source code.

mypy –strict – static type checker. Validates abstract base classes (used for the connector contract), Protocol types (used for the orchestrator API) and Pydantic models. The strict mode forces explicit annotations on every public function.

pytest + pytest-cov – test runner with branch coverage. 244 tests pass at the close of Sprint 3. Coverage exceeds 95% in the central modules (resolver, service, commands, repositories, connectors).

Hypothesis – property-based testing on the Resolver and the inheritance resolution logic.

Table 4.2 summarizes the distribution of tests by type and module.

The same checks run locally and in CI. The local invocation is a single command (`uv run pytest` for tests; `uv run ruff check`, `uv run ruff format` and `uv run mypy` for the others) and the CI runs exactly the same commands. There is no divergence between the developer’s machine and the

Area	Tests	Style
Models (Pydantic)	28	Unit
Resolver	26	Unit + property-based
Connectors (ABC + mock)	39	Unit
Commands	35	Unit + integration
Job engine	12	Integration
Repositories	22	Integration (in-memory TinyDB)
Service (facade)	39	Integration
CLI	30	Unit + integration
REST daemon	13	Integration (ASGI test client)
Total	244	

Table 4.2. Distribution of automated tests at the close of Sprint 3.

CI environment; the only difference is that CI runs in a clean container at every push, eliminating “it works on my machine” problems.

4.4 Continuous integration and delivery

Three GitHub Actions workflows automate the build and release processes. They are stored in `.github/workflows/` and are the single source of truth for what “green” means.

ci.yml runs on every push and every pull request to `main`. It executes the four quality gates of Section 4.3: Ruff lint, Ruff format check, mypy strict and pytest. A failed gate is reported on GitHub and prevents the change from being marked as passing.

publish.yml is triggered by an annotated Git tag of the form `vX.Y.Z`. It builds the multi-stage production image and publishes it to the GitHub Container Registry under two tags: an immutable `vX.Y.Z` (which never changes) and a mutable `latest` (which always points to the most recent release). The choice to use signed Git tags makes the publication explicit: only the maintainer creates a tag, and only that act produces a new image. While the API surface is unstable, the project stays in the `0.x` series.

memoria.yml compiles the LaTeX sources of this report on every push to `main` that touches the `docs/memoria/` directory. It renders the Mermaid diagrams to PDF, compiles the document with

XeLaTeX, uploads the PDF as an artefact and pushes a copy to a dedicated `pdf-preview` branch so that the supervisor always has a stable URL to the latest version.

The end result is that the supervisor or any external reviewer can verify the state of the project at any moment without contacting the author: the public image is downloadable from `ghcr.io`, the source code is on the `main` branch of GitHub, the quality gates are visible in the Actions tab, and the report preview is one click away.

4.5 AI-assisted development

This work was developed with the assistance of Claude (Anthropic), used through both the Claude Code command-line tool and the VSCode extension. We discuss this aspect explicitly because the use of an AI assistant influences how the proof of concept was built and because we believe a TFG that uses such a tool should describe how it does so.

The model and effort level vary with the task. Quick, mechanical tasks (commits, file renames, documentation transcription) use a small and fast model. Routine coding work uses a mid-range model. Critical architectural decisions are taken with the most capable available model and the maximum effort setting. The mapping between phase and model is recorded in the project's `CLAUDE.md` file and is part of the methodology.

The collaboration follows a fixed pattern: *propose, clarify, confirm, execute*. The assistant proposes an approach in plain prose, the author asks clarifying questions or pushes back when the proposal is wrong, the author confirms the path forward, and only then the assistant writes code or modifies files. The author keeps final authority on every decision and is responsible for defending this report; the assistant acts as a structured collaborator, not as a black box. The journal records the entire reasoning of every non-trivial decision exactly as it was discussed, so that any section of the report can be traced back to its origin.

The assistant has access to a persistent context made of versioned files. The entry point is `CLAUDE.md` in the repository root, which imports four files:

- `claude/proyecto.md` – one-page description of the project, scope and constraints.
- `claude/diario.md` – chronological journal of every decision in its context.
- `claude/decisiones.md` – numbered log of technical decisions (DEC-001 to DEC-036 at

the time of writing), each with a back-reference to the journal entry where it was taken.

- `claude/desarrollo.md` – development guidelines: sprints, coding conventions, exception style, docstring format, working agreement with the author.
- `claude/ideas-pendientes.md` – backlog of ideas outside the current scope, classified by area (web GUI, teaching layer, report material, UX).

The four files use Markdown and natural prose so they are equally usable by a human reader: a maintainer who picks up the project after this TFG ends can read `diario.md` as a project history and `decisiones.md` as a design rationale, without needing the AI assistant at all.

4.6 Traceability and knowledge management

The set of `claude/` files is also the traceability backbone of the project. Every numbered design decision in `decisiones.md` points to the journal entry where it was discussed; every journal entry references the decisions taken during that session. The result is that any architectural or implementation choice in the codebase can be traced back to its motivation in a few clicks.

A controlled vocabulary is maintained in `docs/glossary.md`. This file is the single source of truth for the terminology used in the report, in the technical documentation and in the future administrator manual. The glossary distinguishes carefully between terms that look interchangeable but are not (e.g. “descriptor” and “virtual machine” belong to two different layers of the vocabulary; the orchestrator manipulates descriptors, the user sees virtual machines), and records the language convention chosen for each: external vocabulary in user-facing components, internal vocabulary in the code and the technical documents. The glossary of this report is generated from the same source.

The pending-ideas file (`claude/ideas-pendientes.md`) keeps proposals that are out of scope for the current sprint. It is revised at the start of every sprint to decide what enters next and what can be promoted to the report’s future-work section. This prevents valuable ideas from being lost between sprints without mixing them with the active scope.

Three external reviewers (a hypothetical maintainer, the supervisor and the tribunal) would all benefit from the same traceability: they can independently audit how a decision was reached without having to ask the author. We consider this an explicit deliverable of the methodology, not a side effect.

DRAFT

5

Development, testing and results

This chapter is the technical core of the report. It walks through the development cycle of the proof of concept in five blocks: requirements analysis, architectural design, implementation, testing strategy and functional demonstration. It closes with a qualitative comparison against the two libraries the supervisor had written before this work. The detailed requirements catalogue is moved to [Appendix B](#); the version presented here is the structured summary that supports the rest of the chapter.

5.1 Requirements analysis

The requirements analysis was carried out as Phase 2 of the project and produced `docs/requirements.md` in the repository. The analysis follows a lightweight format suited to a proof of concept: actors and use cases in natural language, functional requirements grouped by area, a short table of non-functional requirements, and an explicit list of items that are out of scope for v1. We summarize the structure here; the full catalogue is in [Appendix B](#).

5.1.1 Actors

Table [5.1](#) lists the actors of the system. The role design follows DEC-011 and DEC-021: three roles with cascade inheritance through the tree, and a single user that can hold different roles in different branches.

Actor	Description
Administrator (superuser)	Manages hypervisors, base templates, users, the full tree and global configuration.
Manager	A professor or equivalent. Manages assigned branches of the tree. Uses both the core and the teaching application.
End user	A student or equivalent. Operates only the VMs assigned to them.
Hypervisor	External system. UnivOrch talks to it through a connector; the hypervisor does not initiate actions.
System	UnivOrch itself, running the job engine and automatic tasks (backup rotation, retention cleanup).

Table 5.1. Actors of UnivOrch.

5.1.2 Use cases

Use cases are split into three groups. *Core* use cases are generic operations of the orchestrator (authentication, defining hypervisors and templates, creating folders, deploying and undeploying VMs, viewing job history). *Teaching application* use cases (UC-TA-1...) operate at the domain level: deploying a whole subject from a roster of students, regenerating a student's environment, end-of-term recycling. *System* use cases describe the automatic behaviour of the daemon (backup, retention cleanup, detection of jobs left in flight after a crash). The full catalogue contains 32 use cases.

The most representative use cases of the core are UC-ADM-2 (define a hypervisor with its credentials and datastores), UC-ADM-3 (define a base template), UC-MGR-1 (a manager creates folders inside their branch), UC-MGR-3 (a manager loads a YAML definition of a group of VMs), UC-EU-1 (an end user operates their own VMs through the simplified view) and UC-ADM-8 (the administrator force-undeploys a descriptor stuck in **broken** state).

5.1.3 Functional requirements

The functional requirements are grouped by area: descriptor tree and hierarchical organization; declarative load and inheritance; hypervisor connectors and life-cycle operations; jobs and operations; persistence and backup; identity, authentication and authorization; and the teaching application. Within each area, the requirements stay close to the use cases of the same area and introduce only the precision

needed to bridge into the architecture phase.

5.1.4 Non-functional requirements

Table 5.2 lists the headline non-functional requirements. The proof of concept is not a benchmark exercise; the numbers below are soft targets used to keep the implementation honest, not contractual service levels.

Quality	Target
Reproducibility	A new clone of the repository plus one command must start a working development environment.
Portability	The same code base must support new hypervisors without changes to the core.
Testability	The core must be exercisable without a real hypervisor.
Maintainability	Every component must have a test, every public function must be typed, and the documentation must trace every architectural choice back to a numbered decision.
Operability	The complete service must run inside a single Docker container and be deployable in one command.

Table 5.2. *Non-functional requirements of UnivOrch v1 (summary).*

5.1.5 Out of scope for v1

The following items are explicitly excluded from this proof of concept: automatic reconciliation loops, active/passive high availability, IPAM integration, snapshot management through the orchestrator, discovery of pre-existing VMs not created by UnivOrch, multi-tenant isolation beyond what RBAC provides, and any integration with single-sign-on or directory services.

5.2 Architectural design

The architecture is documented in detail in `docs/architecture.md` and visually summarized in `docs/diagrams.md`; this section presents the same content in narrative form, with the diagrams embedded next to the text they support.

5.2.1 Two-layer architecture (DEC-004)

UnivOrch is built as two clearly separated layers. The lower layer is a *generic orchestration engine* that knows about folders, descriptors, jobs, connectors and inheritance, but knows nothing about academic subjects, students or any other domain. The upper layer is a *domain application* that translates the vocabulary of a specific use case into operations of the engine. The teaching application of this TFG is the only upper layer implemented; Fig. 5.1 also shows where alternative upper layers would sit (a CTF organizer, a workshop deployer, a research-lab orchestrator).

The boundary between layers is the public REST API of the daemon. Every upper layer is a client of this API. The same separation applies to the operational interfaces (CLI, future web GUI): they are also clients of the API, not parts of the engine. Nothing in the engine knows what kind of client is calling it.

5.2.2 C4 context and containers

Fig. 5.2 shows the system at C4 level 1. Three kinds of human or external actor interact with UnivOrch: the administrator or teacher who defines the tree and triggers deployments; the student who operates their own machines; and external integrations (CI/CD pipelines, GitOps tools) that consume the public REST API. The orchestrator itself talks to the underlying hypervisors (VMware vSphere, Proxmox VE), and students access their VMs by IP *directly*, not through the orchestrator – this distinction matters when discussing the failure modes.

Fig. 5.3 zooms into the C4 level 2 view. There are two clearly delimited deployable units inside the system: the *daemon* `univorchd` (a Python process running FastAPI on Uvicorn) and the *CLI* `univorch` (a Python program based on `cmd2` that talks to the daemon by HTTP). Both live in the same package distribution and are installed by the same Docker image; one runs as a long-lived service, the other as an on-demand command. The future web GUI fits in this view as a third container, with the

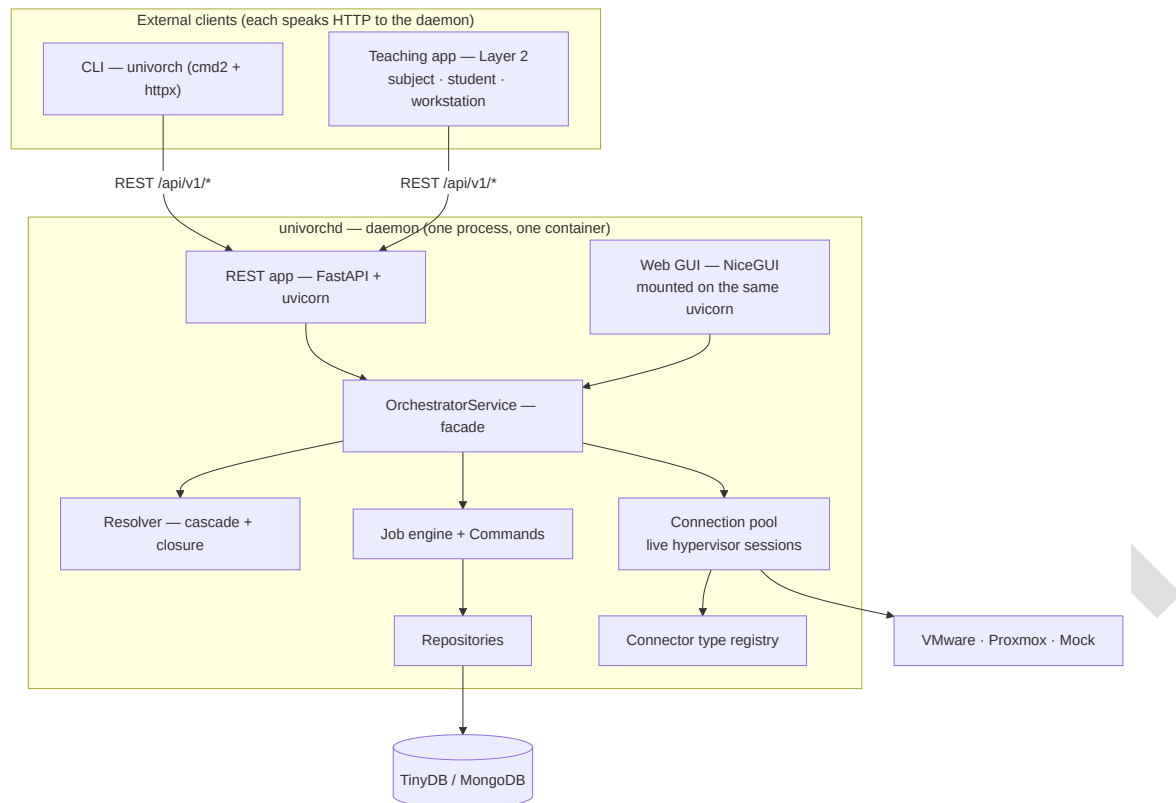


Figure 5.1. Two-layer overview: the daemon *univorchd* hosts the core; every client (CLI, web GUI, teaching app) speaks REST to it.

same role as the CLI – a thin client of the REST API.

5.2.3 Deployment view

Fig. 5.4 shows the deployment view of Sprint 3. The target deployment is a single Linux host with Docker that runs the *univorchd* container. The container exposes one HTTP port (default 8080) for the REST API and mounts a Docker named volume (*univorch_data*) where the TinyDB JSON file lives. Both the named volume and the port are configurable through environment variables.

The clients sit on tier 1, on any host that can reach the daemon port. The hypervisor hosts are tier 3; the daemon talks to them through the connectors implemented in the engine. In the mock-driven demo, tier 3 is not used: the mock connector lives inside the daemon and provides a fictitious hypervisor in memory.

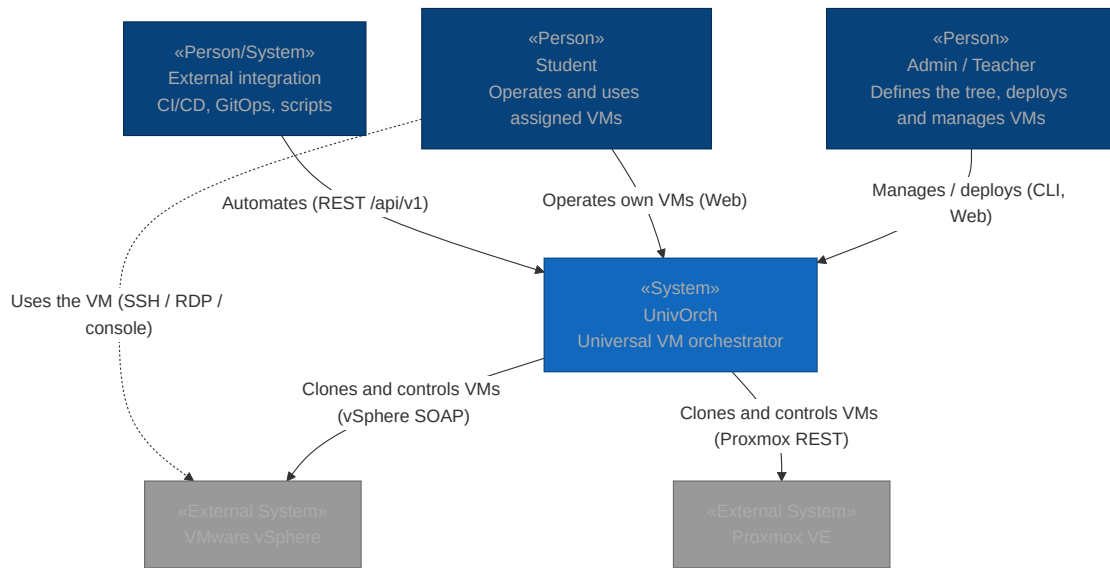


Figure 5.2. C4 level 1 (Context): people and external systems around UnivOrch.

5.2.4 Components (as-built)

Fig. 5.5 is the C4 level 3 view, restricted to what was actually built. The diagram uses a legend with two states: *implemented* (solid green) for everything delivered up to Sprint 3, and *pending* (dashed grey) for everything that was designed but is left as future work. The pending blocks include the web GUI, the real hypervisor connectors (VMware and Proxmox) and the teaching application.

Inside the daemon, the implemented components are: the REST layer (`create_app`, the FastAPI application, the `HttpClient` that lives in the CLI), the `OrchestratorService` facade, the lexical-closure resolver with its generic walker (`_find_resource`), the connection pool indexed by folder path, the connector type registry (`CONNECTOR_TYPES`), the YAML parser, the job engine with its `Commands`, the three TinyDB repositories (folders, descriptors, jobs), the abstract `HypervisorConnector` and the `MockConnector`. The diagram makes the layered structure explicit: the REST layer depends only on the facade, the facade depends on the resolver, the job engine, the pool and the parser, the job engine depends on the connector abstraction and on the repositories, and the repositories depend on TinyDB.

5.2.5 Domain model

The descriptor tree is implemented as a flat collection of records indexed by their *materialized path* (DEC-008). A folder at `/lab/networks` is one record, the descriptor `student01` inside it is another

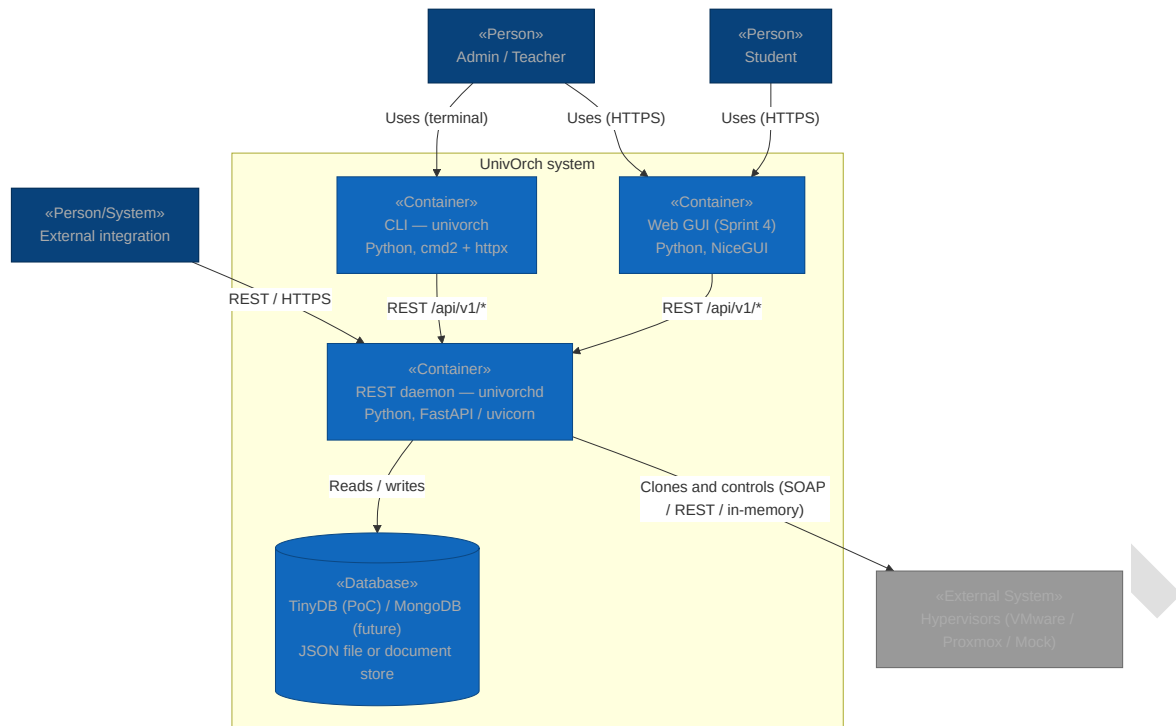


Figure 5.3. C4 level 2 (Containers): independently runnable units. The daemon *univorchd* and the CLI *univorch* are two separate binaries; every client speaks HTTP.

record at `/lab/networks/student01`, and so on. Navigating the tree upward or sideways is then a matter of string operations on the path; there are no pointers between records to maintain. This choice keeps the TinyDB schema flat and the repository operations trivial.

Inheritance of properties follows DEC-026: the rule depends on the data type of the field. A scalar field (`cpu`, `memory_mb`) is overridden by the child; a list field (`imports`) accumulates from parent to child; a dictionary field (`hypervisors`, `vm_templates`) is recursively merged. The merge stops at the explicit boundary set by the `import:` field of each folder, which decides what names from the parent are visible inside this folder.

The template lexical closure deserves special mention. When a template is defined in folder *A* and imported into folder *B*, any reference inside the template (e.g. which hypervisor to target) is resolved starting from *A*, not from *B*. The template carries its definition environment with it. This is the behaviour of a lexical closure in a programming language and is the refinement of DEC-012 applied during Sprint 2.

Fig. 5.6 shows the model classes. There are two families of types. The first family is the *persisted* types (`Folder`, `Descriptor`, `Job`); they carry an absolute path or identifier and live in TinyDB. The second family is the *document* types (`DefinitionDocument`, `FolderDef`, `DescriptorDef`,

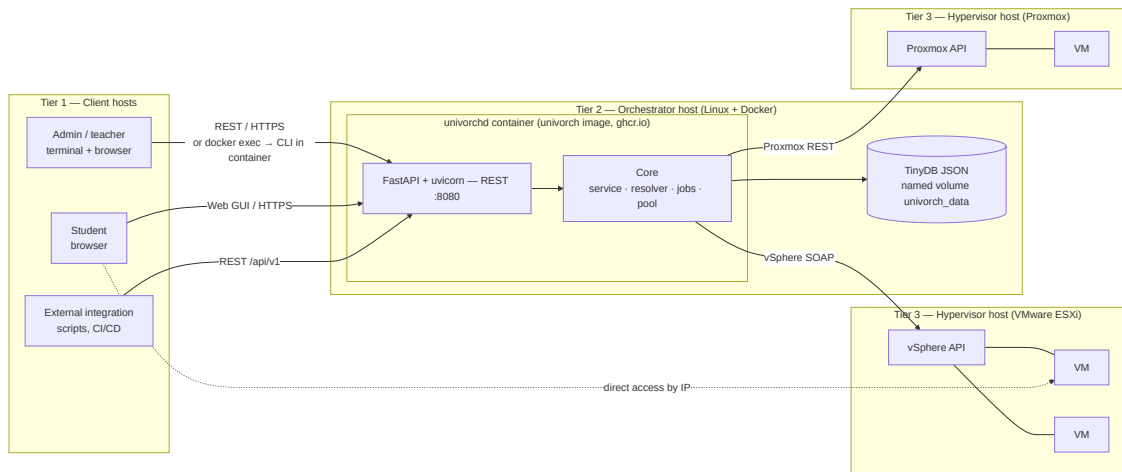


Figure 5.4. Deployment topology since Sprint 3: clients on tier 1, the daemon container plus its named volume on tier 2, and the hypervisor hosts on tier 3.

HypervisorDef, VMTemplateDef); they are produced by the YAML parser and travel into the load operation, where the engine turns them into persisted entities. The split is intentional: documents allow relative paths and aliases for the YAML syntax (use `hypervisor:`, `define templates:`), persisted entities use canonical absolute paths.

5.2.6 Patterns applied

Several classical design patterns shape the implementation. The relevant ones are listed below; the citation of each pattern is left to the references in `docs/architecture.md` to keep the prose tight.

Facade (DEC-031) – the `OrchestratorService` class is the single entry point that every client uses. It encapsulates the engine and protects clients from the internal repositories, the job engine, the connector pool and the resolver. The REST layer talks to the facade only.

Command pattern (DEC-028) – every state-changing operation is encapsulated in a `Command` class (`DeployCommand`, `UndeployCommand`, `StartCommand`, ...). Commands are validated before being run; if validation passes, the job engine executes them and tracks the outcome in the job log.

Repository (DEC-007/030) – persistence is hidden behind a repository class for each aggregate root (folder, descriptor, job). The repositories take and return Pydantic models, not TinyDB documents. Replacing TinyDB with MongoDB requires rewriting only the repository classes.

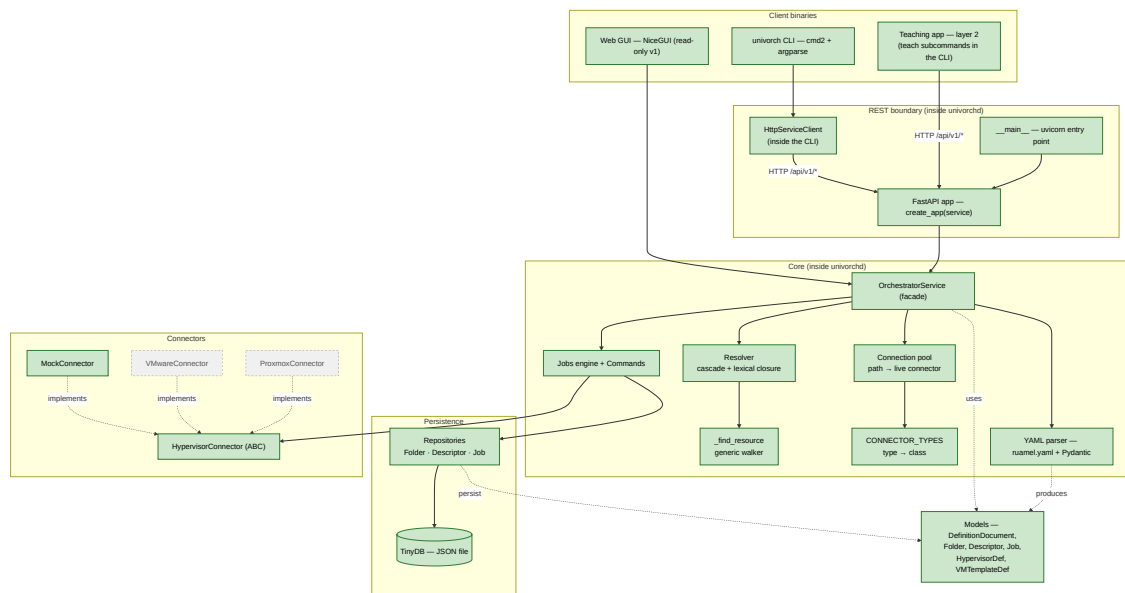


Figure 5.5. C4 level 3 (Components) modules inside *univorchd* after Sprint 3. Solid green = implemented; dashed grey = designed, not yet implemented.

Strategy / Abstract Base Class (DEC-029) – the hypervisor connector is an abstract base class with a small and stable interface. The mock connector implements it in memory; the future VMware and Proxmox connectors will implement it against their respective APIs.

Protocol (Sprint 3.2) – the boundary between the CLI and the daemon is described by the `OrchestratorAPI` Protocol. Two classes implement it structurally: the in-process `OrchestratorService` and the `HttpServiceClient`. Clients (the CLI in particular) depend on the Protocol, not on either concrete class.

Fig. 5.7 shows how these classes relate. The `OrchestratorAPI` Protocol unifies the in-process and the HTTP paths; the engine sits behind the facade; the Commands depend on repositories and on the connector pool.

5.2.7 Declarative load flow (DEC-027)

The system follows a strict declarative model. The user does not issue create or delete commands; they describe what they want by loading a YAML `DefinitionDocument`, and the engine works out the operations needed to make the tree match the document. The load operation goes through the five stages of Fig. 5.8:

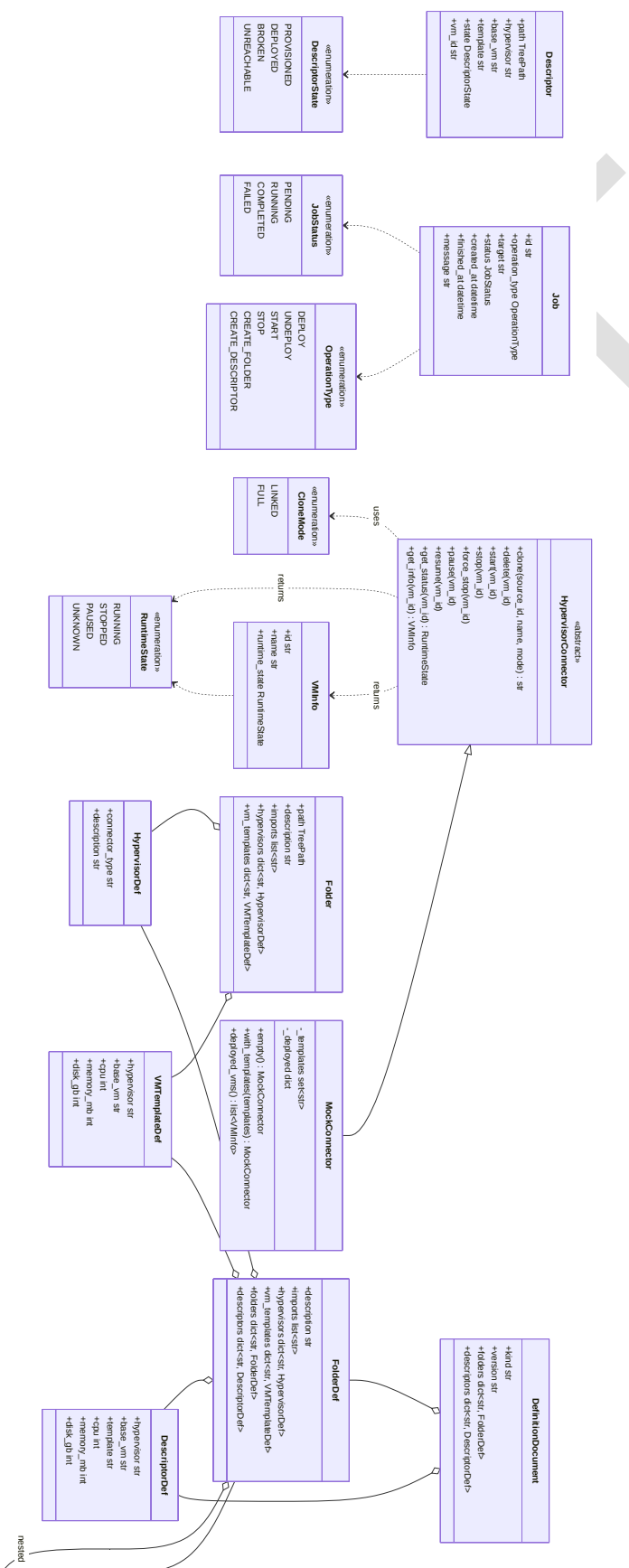


Figure 5.6. Domain model and connector contract (C4 code level, view 1/2). Persisted entities (*Folder*, *Descriptor*, *Job*), input documents (*DefinitionDocument* and friends) and the abstract *HypervisorConnector*.

1. **Parse.** The YAML is read with `ruamel.yaml` and validated by Pydantic against the document schema. Syntax errors are caught here.
2. **Diff.** The engine compares the document against the current tree. The result is a list of intended operations (create this folder, add this descriptor, update this template) without yet touching anything.
3. **Validate.** Every intended operation is validated: does the user have permission, is the hypervisor name resolved against an existing definition, is the target datastore present? Validation *rejects* the load when any check fails, *before* any job is created.
4. **Plan.** The same flow with execute disabled. Useful as a dry-run command. Returns the intended operations and the validation result without touching the tree.
5. **Execute.** The operations are translated into Commands and submitted to the job engine. Each job is persisted, executed and logged.

The strict separation between validation and execution is a deliberate choice (DEC-032). A failed validation never creates a job; this keeps the job log free of records that exist only to mark “user submitted a bad request.” Inside execution, however, a job that fails after passing validation *does* create a record, because that is exactly the kind of incident the operator needs to see.

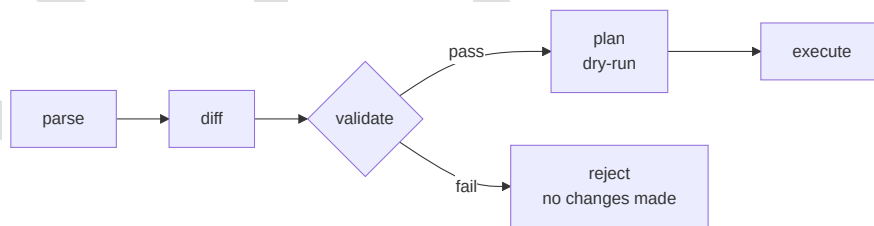


Figure 5.8. Stages of *load / plan*: validation rejects before any state change.

5.2.8 Descriptor state machine (DEC-022, DEC-032)

Every descriptor has a state, tracked by the orchestrator independently of the runtime state of the underlying VM. There are four states: **provisioned** (the descriptor exists but no VM has been created in the hypervisor yet), **deployed** (the descriptor matches a real VM in the hypervisor), **unreachable** (the hypervisor cannot be contacted; the actual state of the VM is unknown) and **broken** (an operation

failed in a way that leaves the orchestrator and the hypervisor out of sync). The transitions are shown in Fig. 5.9.

Two characteristics of this state machine deserve emphasis. First, *state changes only as the result of a completed job*: there is no background polling that silently transitions a descriptor. This makes every state change auditable through the job log. Second, the runtime state of the VM (running, stopped, paused) is reported by the hypervisor on demand and not persisted by the orchestrator. This keeps the orchestrator's model small and prevents drift between two state representations that would inevitably diverge.

The **broken** state has the precise definition added in DEC-032: a descriptor is **broken** only after a job that was expected to converge the descriptor with the hypervisor (typically a deploy or undeploy) failed in a way that left a VM partially created or partially destroyed. The recovery path is the administrator's **force-undeploy** operation: clean up whatever exists in the hypervisor and return the descriptor to **provisioned**.

5.2.9 REST as the public boundary

The decision to expose a REST API as the boundary of the daemon was made in Sprint 3 and deserves an explicit defence because it shapes the entire client architecture. The argument for REST is *not* that the CLI needs to be usable remotely without SSH; the administrator already has SSH. The argument is that the REST API exists as a *public boundary for external integrations*. External clients (CI/CD pipelines, GitOps tools, future teaching applications, third-party scripts) have a clean and documented contract to depend on. The CLI of UnivOrch then *also* uses that contract, which removes any temptation to add a back door for local use and keeps the in-process and out-of-process paths identical.

An intermediate design (the CLI talks to the engine in-process when running on the same host and over HTTP when remote) was considered and discarded during Sprint 3. The advantage of fewer moving parts was real, but the cost in duplicated logic and divergent behaviour between the two paths was higher. The decision aligns UnivOrch with mature peers: **docker**, **kubect1** and **podman** all run as thin HTTP clients of a daemon, even when both live on the same machine.

5.3 Implementation

This section presents the implementation by layer rather than by sprint. The sprints are documented in Chapter 4; for the description of the product, the layered view is clearer. The code lives under `src/univorch/`; each subsection below corresponds to one subdirectory of that tree.

5.3.1 Orchestrator core

The core consists of the Pydantic models, the resolver, the job engine, the repositories and the facade service.

Models. The domain types are implemented as Pydantic `BaseModel` classes with strict configuration (`extra="forbid"`, field validators for the path syntax). The two families discussed in Section 5.2.5 are kept in separate modules to avoid accidental cross-contamination between the persisted form and the document form.

Resolver. The resolver is a pure function: given a folder path and the current tree, it returns the effective definition of that folder, with cascade inheritance applied and the lexical closure resolved for any imported template. “Pure” has a precise meaning here: no side effects, no I/O, no exceptions other than for truly invalid input. This makes the resolver the ideal candidate for property-based testing with Hypothesis (Section 5.4). The inheritance rule is encoded once per data type (scalar/list/dictionary) and the same rule applies to every field. The walker that follows the import chain through ancestor folders is generic, parameterized by the resource type (template, hypervisor); adding a new inheritable resource type in the future is a one-line change.

Job engine and Commands. The job engine accepts a `Command` and turns it into a `Job`. The `Command` is responsible for *validation* (a single method that returns a list of human-readable error messages and does not touch the database) and *execution* (a single method that mutates the world). The engine wraps the `Command`, creates a `Job` record in TinyDB, transitions the descriptor state if needed, captures exceptions and records the outcome. Six `Commands` are implemented in v1: `DeployCommand`, `UndeployCommand`, `StartCommand`, `StopCommand`, `CreateFolderCommand` and `CreateDescriptorCommand`.

Repositories. There is one repository per aggregate root (folder, descriptor, job). The interface is small: `save`, `get`, `exists`, `subtree` or `find_by_status`, `delete`. The TinyDB implementation operates on Pydantic models through `model_dump()` and `model_validate()`. A different backend

(MongoDB, PostgreSQL) would replace the body of these methods without changing any caller.

Facade service. The `OrchestratorService` class is the public face of the engine. It exposes one method per public operation (`deploy`, `undeploy`, `start`, `stop`, `status`, `list_tree`, `inspect`, `load`, `folder_exists`) and resolves the connector session through the connection pool. The facade is the only place where coordination between repositories, the resolver, the job engine and the connector pool happens.

5.3.2 Mock connector

The mock connector is the only implementation of the `HypervisorConnector` abstract base class delivered in v1. It keeps its state in memory (a dictionary of fake VMs, a counter of created VM identifiers, a list of pre-loaded demo templates) and implements every method of the contract with the same observable behaviour as a real hypervisor. The mock is not a stub: it actually clones the requested template into a new VM record, transitions its runtime state under `start` and `stop` and reports the right value under `get_status`. The whole system runs end-to-end against this connector, without any real hypervisor; this is what makes the demo of Section 5.5 reproducible on any laptop.

5.3.3 CLI interface

The CLI is implemented with `cmd2`, which provides both a batch mode (one command at a time, suitable for shell scripts) and an interactive REPL (with history, tab completion and built-in `help`). Argument parsing for every command uses the `argparse` integration of `cmd2`; this gives auto-generated, coloured help for free.

The CLI does not embed the engine. It accepts an instance of `OrchestratorAPI` at construction time and forwards every command to it. By default the instance is an `HttpClient` configured against the daemon's URL; a test that wants to drive the CLI without a daemon passes an in-process `OrchestratorService` instead. The `HttpClient` translates HTTP transport errors (connection refused, timeout, 4xx response) into `OperationError` exceptions with actionable messages, so the user does not see HTTP-level error codes in the terminal.

5.3.4 REST daemon

The daemon is a FastAPI application served by Uvicorn. The application is built by `create_app(service)`, which takes an instance of the facade and registers one endpoint per public operation. Paths are passed as query parameters (`?path=/lab/student01`) so they remain easy to read in URLs and easy to test from the command line with `curl`. Error codes follow a small and explicit convention: 200 OK for successful operations, 400 Bad Request for failed validation (returning the list of error messages), 422 Unprocessable Entity for malformed inputs that fail Pydantic validation before reaching the service. There is a `GET /api/v1/health` endpoint for liveness checks.

The daemon entry point lives in `src/univorch/interfaces/rest/__main__.py` and is registered in `pyproject.toml` as the `univorchd` console script. It composes a real `OrchestratorService` (with TinyDB repositories on the path declared by the environment variable `UNIVORCH_DB_PATH`) and runs Uvicorn on the port declared by `UNIVORCH_PORT`.

5.3.5 Web interface (read-only)

A first version of the web interface is delivered in Sprint 4 as a read-only view of the tree. It is built with NiceGUI and mounted on the same Uvicorn process as the REST application via `ui.run_with(fastapi_app, ...)`: a single port serves both `/api/v1/*` and the web pages. This is a deliberate departure from the Sprint 3 “every client is HTTP” rule: the web is part of the daemon, not an external HTTP client of it. The external clients (CLI, the teaching subcommands, third-party integrations) keep going through the REST API.

The v1 has three pages. The index page shows a header, four summary cards (*total VMs*, *provisioned*, *deployed*, *broken or unreachable*) and an expandable tree of folders and descriptors. The descriptor nodes carry an icon and a badge that match the state vocabulary of the CLI (grey for *provisioned*, green for *deployed*, red for *broken*, yellow for *unreachable*). Clicking a descriptor node navigates to the descriptor detail page, which calls `service.inspect(path, resolved=True)` and `service.status(path)` and renders the eleven relevant fields of the effective definition. Folder nodes only expand or collapse, with no navigation.

Write operations (deploy, start, stop, undeploy, load), authentication and the workstation/desk view for students remain on the roadmap; the read-only v1 is the proof that the boundary with the orchestrator service works correctly through a graphical client.

5.3.6 Teaching application (layer 2)

The teaching application is the only layer-2 application implemented, and it is the practical demonstration that the two-layer split (DEC-004) works: a complete domain workflow built entirely on the generic engine, without the engine learning any teaching vocabulary. It lives in `src/univorch/teaching/` and is exposed as a `teach` subcommand group of the CLI; it talks to the core through the same `OrchestratorAPI` as every other client.

The only change it required in the core is a single opaque `metadata` dictionary on the folder model (DEC-038). The core stores it and never interprets it; the teaching layer puts `{kind: subject, desktop: [...]}` there to mark a folder as a subject and to record which templates make up a student's set of machines. A different layer-2 application would reuse the same field for its own markers, so the core stays domain-agnostic.

The workflow has two inputs and a handful of commands:

teach load-subject validates that a YAML document is a well-formed subject (marked `kind: subject`, non-empty desktop, every desktop template resolvable, no child folders) and, only then, loads it with the core `load`. A malformed subject, or the wrong file altogether, is rejected with a single actionable message before anything is created.

teach load-students reads the subject's desktop from its metadata and generates, under the subject, one folder per student (named after the username) and one descriptor per desktop template inside each. Each descriptor carries `use template: <name>` and the student folder carries `import: *`, so the descriptors resolve the subject's templates by cascade — the generated tree is a normal core document the resolver already understands. The operation is add-only: existing students are no-ops, none are removed (DEC-039).

teach save-students exports the current roster, the inverse of `load-students` at the list level.

Two design choices keep the layer thin. First, there is no new `use desktop:` keyword: because the application creates the descriptors, it writes `use template: <name>` directly, which the core already resolves. Second, the operations are decoupled from the CLI into a `teaching/operations.py` module, so they are tested against a real in-process service without a daemon. An end-to-end demonstration — load a subject of six machines, load a roster of ten students (one of them an email address), and deploy one of the resulting machines — runs against the mock connector and produces sixty provisioned descriptors that each resolve their template by inheritance.

Deploying a whole subject in one batch command, removing students that left the roster, email notifications and a dedicated teaching web view are designed and left as future work.

5.3.7 Packaging and distribution

The production image is built from a multi-stage Dockerfile. The first stage uses `uv` to resolve and install only the runtime dependencies. The second stage is a slim Python 3.12 base that copies the resolved virtual environment, the application code and the entry points. The resulting image is around 200 MB and takes a few seconds to pull. The image is published to `ghcr.io/clamaveruma/univorch` by the `publish.yml` workflow, triggered by signed tags of the form `vX.Y.Z`.

A `docker-compose.yml` file declares the service, the named volume (`univorch_data`) where the TinyDB file lives, the port mapping and the environment variables. A thin shell wrapper, `univorch.sh`, exposes a convenient command surface (`start`, `stop`, `restart`, `status`, `logs`, `cli`).

For new users, an `install.sh` script is published from a fixed URL and does the whole installation in one command. The script detects the local Docker version, picks a free port if 8080 is taken, falls back from `docker compose` to `docker-compose v1` if needed and explains what it is going to do before doing it. The script is documented in the supervisor's tutorial (`docs/tutorial-profesor.md`).

5.4 Testing strategy

The testing strategy is a small but consistent application of the test pyramid. Most tests are unit tests; a strong layer of integration tests covers the facade and the REST endpoints; a minority of tests use property-based testing to exercise the resolver. There are no end-to-end tests against a real hypervisor in v1, because no real connector exists yet; the demo of Section 5.5 is the closest analogue.

5.4.1 Unit tests

Unit tests target a single class or module. They are fast (the full unit suite runs in well under one second) and they pin behaviour at the granularity needed to refactor with confidence. The Pydantic models are tested for round-trip serialization and for field validation. The Commands are tested with both a successful and a failing case for each branch of their validation. The mock connector is tested

by exercising every method of the `HypervisorConnector` contract.

5.4.2 Integration tests

Integration tests target the facade and exercise it through the complete chain: facade → resolver → job engine → repositories → TinyDB → mock connector. The repositories are configured against an in-memory TinyDB so the tests stay fast and isolated from disk; the mock connector is used in its `empty()` variant so every test starts from a known state. The integration tests are the ones that catch interaction bugs between pieces that look correct in isolation.

5.4.3 Property-based tests

The resolver is a pure function over plain data. Property-based testing with Hypothesis is well suited to it. The strategy generates random trees of folders and descriptors with random imports, templates and hypervisor definitions, and checks properties that should hold for any input:

- Idempotence: resolving the same descriptor twice returns the same result.
- Closure: a template imported from *A* resolves its internal references from *A*, regardless of the folder where the import happens.
- Bounded import: a folder cannot see a resource that no ancestor explicitly imported.

Hypothesis shrinks failing cases to a minimal counterexample automatically; in practice this has caught two subtle bugs in the inheritance resolution during the development of Sprint 2.

5.4.4 REST tests

The REST endpoints are tested with the `httpx.AsyncClient` of FastAPI, which routes requests directly to the ASGI application in-process. This catches client/server divergences (a missing field in the request schema, a wrong status code) without spinning up a real Uvicorn server. The same tests also exercise the `HttpServiceClient` of the CLI: the client is constructed against the in-process test transport, which keeps the test fast and deterministic. This is one of the cleanest benefits of the Sprint 3.2 decision to make the CLI a pure HTTP client: the same test suite covers the wire format and the client code at once.

5.4.5 Coverage and metrics

Table 5.3 reports the line and branch coverage at the close of Sprint 3, measured by `pytest-cov` on the central modules. Coverage is above 95% in every module that contains domain logic; the lower number on the CLI module reflects mostly interactive code paths and the launch wiring that the test suite intentionally does not exercise.

Module	Line coverage	Branch coverage
<code>models.py</code>	100%	100%
<code>resolver.py</code>	100%	100%
<code>service.py</code>	97%	95%
<code>jobs/commands.py</code>	100%	100%
<code>jobs/engine.py</code>	100%	100%
<code>persistence/tinydb/</code>	100%	100%
<code>connectors/mock.py</code>	100%	100%
<code>interfaces/cli/</code>	92%	88%
<code>interfaces/rest/</code>	95%	90%

Table 5.3. Coverage on the core modules (286 tests passing, including the teaching layer).

5.5 Functional demonstration

This section describes the end-to-end demonstration that runs against the mock connector, without any real hypervisor. The same demonstration is in `demo/README.md` of the repository, with copy-pasteable commands. Screenshots will be added before the final submission of the report; provisionally the demonstration is described in prose.

5.5.1 Starting the daemon

The administrator pulls the public image and starts the service in one command:

```
docker run -d --name univorch -p 8080:8080 \
-v univorch_data:/data \
```

```
ghcr.io/clamaveruma/univorch:latest
```

A liveness check at `http://localhost:8080/api/v1/health` returns `{"status":"ok"}`. The OpenAPI specification is auto-generated by FastAPI and available at `/api/v1/openapi.json`; the Swagger UI is at `/api/v1/docs` and is the easiest way to explore the API without writing any code.

5.5.2 Loading the demo tree

The demo tree is a small YAML file describing one administrative folder `/lab` (with the hypervisor definition and a base template), one folder `/lab/networks` (the subject) and three descriptor entries `student01`, `student02`, `student03` that use the imported template:

```
kind: definition
version: "1"
folders:
  lab:
    description: "Computer Science Lab"
    define hypervisors:
      mock01: { type: mock }
    define templates:
      linux-vm:
        use hypervisor: mock01
        base_vm: linux-base
    folders:
      networks:
        description: "Computer Networks course"
        import: [linux-vm]
        descriptors:
          student01: { use template: linux-vm, cpu: 2 }
          student02: { use template: linux-vm, cpu: 2 }
          student03: { use template: linux-vm, cpu: 4 }
```

The administrator loads the document into the orchestrator:

```
univorch load demo/setup.yml /
```

The CLI prints the list of operations performed, one line per folder or descriptor created. After this command, `univorch tree /` renders the descriptor tree with state glyphs next to each node: `[ ]` for provisioned, `[*]` for deployed, in red for broken, in yellow for unreachable.

5.5.3 Inspecting the inheritance

The `inspect` command shows the effective definition of any folder or descriptor with the inheritance applied. Three modes are supported. `--local` prints only what is written at the node itself, with no merging. `--expanded` prints the result of applying imports and cascade inheritance but without following references inside templates. The default mode prints the fully resolved definition: the hypervisor name, the base VM, the hardware parameters and any local overrides.

This is how the demonstration shows the lexical closure: the descriptor `student03` declares no hypervisor and no base VM explicitly; `inspect` reveals that it inherits the template `linux-vm` (defined in `/lab`, not in `/lab/networks`) and that the template itself resolves `use hypervisor: mock01` against the same folder `/lab` where it was defined. The override of `cpu` to 4 appears explicitly as a local property.

5.5.4 Full life cycle

The administrator runs the complete life cycle on one of the descriptors:

```
univorch deploy /lab/networks/student01
univorch start  /lab/networks/student01
univorch status /lab/networks/student01
univorch stop   /lab/networks/student01
univorch undeploy /lab/networks/student01
```

Each command produces a Job that is persisted in the daemon. The `deploy` job transitions the descriptor from `provisioned` to `deployed`; the mock connector records the creation of a new VM with a fresh identifier. The `start` job transitions the runtime state of the VM but does not change the descriptor state. The `status` command reads both states from the connector. The `stop` and `undeploy` jobs reverse the sequence; after `undeploy`, the descriptor is back in `provisioned` and the mock connector has no record of the VM.

5.5.5 Exercising the REST API

The same operations can be invoked directly through the REST API with `curl`:

```
curl -X POST 'http://localhost:8080/api/v1/deploy?path=/lab/networks/student01'  
curl 'http://localhost:8080/api/v1/status?path=/lab/networks/student01'
```

This is the same code path the CLI uses, and it is also the entry point that future external integrations (a CI pipeline, a teaching front-end, a monitoring agent) would use. The Swagger UI displays every endpoint with its parameter schema, response schema and example values; an external developer who has never seen UnivOrch can navigate it in a few minutes.

5.6 Comparison with `esxobjects` and `yamlinfr`

The two libraries written by the supervisor before this work, `esxobjects` and `yamlinfr`, are the operational precedent of UnivOrch. This section compares them with UnivOrch on the same axes that matter for a proof of concept aimed at maintainability and reuse. Table 5.4 summarizes the comparison.

The comparison must be read honestly. The two previous libraries *solved their problem* – a real working deployment system that the supervisor and his students have used for actual courses. UnivOrch is, today, a proof of concept of a more ambitious model: it generalizes the hypervisor coupling, lifts the deployment script into a long-running service, formalizes the data model into a tree with inheritance, exposes a public REST contract and adds the infrastructure (test discipline, CI/CD, packaged distribution) that the previous libraries did not have time to develop. It does *not yet* replace them operationally: the real VMware connector is still to be implemented, so an actual deployment against the teaching lab cannot run on UnivOrch today.

The intended path forward is to write the VMware connector against the abstract interface defined in Sprint 1, validate it against the same lab that `yamlinfr` drives today, and then deprecate `yamlinfr` in favour of UnivOrch with the teaching application of Sprint 5 on top. This work is outside the scope of this TFG, but the present proof of concept proves that the model underneath is sound.

Axis	esxobjects	yamlinfr	UnivOrch
Scope	VMware vSphere API wrapper.	YAML-to-deployment script over esxobjects + external IPAM.	Long-running orchestration service, hypervisor-agnostic.
Hypervisor coupling	VMware only.	VMware only (via esxobjects).	Pluggable: mock today, VMware and Proxmox planned.
Model	Object-oriented wrapper around vSphere objects.	Flat YAML, one infrastructure per file.	Tree of folders with cascade inheritance and lexical closure of templates.
Persistence	None: each invocation is independent.	None: stateless deployment.	TinyDB document database (replaceable by MongoDB).
Public boundary	Python library.	Command-line script.	REST API with OpenAPI specification.
Test coverage	Limited.	Limited.	286 tests, >95% coverage on the core.
Distribution	Internal department use.	Internal department use.	Public Docker image in ghcr.io , single-command install.

Table 5.4. Comparison of *esxobjects*, *yamlinfr* and *UnivOrch*.

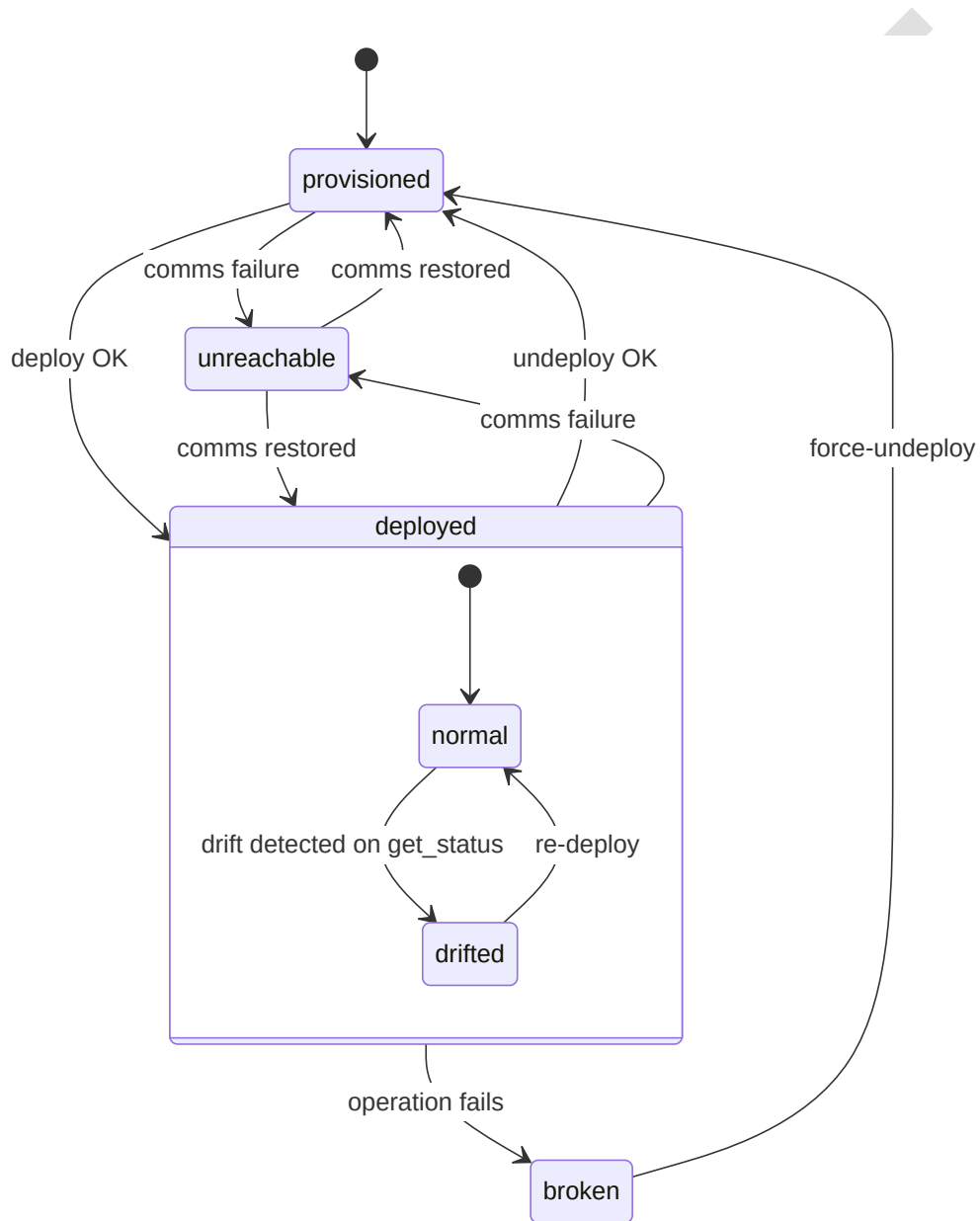


Figure 5.9. Descriptor state machine. Runtime states (running, stopped, paused) belong to the hypervisor and are not persisted by the orchestrator.

DRAFT

6

Conclusions and future work

This chapter closes the report. It reviews the proof of concept against the objectives set in Chapter 1, summarizes the main contributions and the known limitations, and outlines the natural extensions of the work beyond this TFG. The chapter is short on purpose: most of the underlying material has already been defended throughout the report, so the conclusions can stay tight.

6.1 Conclusions

6.1.1 Fulfilment of objectives

The seven objectives declared in Section 1.2 are reviewed here against what was actually delivered. Table 6.1 summarizes the assessment; the following paragraphs add the qualifications that the table cannot capture.

Objectives O1 through O6 are met by the delivered system: the engine runs, the declarative model with cascade inheritance and template closure works end-to-end, the connector contract is defined and exercised by the mock implementation, the REST API exposes every public operation, the CLI is a pure HTTP client of the same API, and the engineering practices (test-driven development, strict static typing, continuous integration, packaged distribution) hold across the whole project.

Objective O7 is marked *partial*: every extension point is designed and most of them are wired in code, but the next deliverables that depend on them (real connectors, web GUI, teaching application)

ID	Objective	Status	Reference
01	Hypervisor-agnostic orchestrator core	Delivered	Sections 5.2 and 5.3
02	Declarative model with inheritance	Delivered	Sections 5.2.5 and 5.2.7
03	Connector contract for new hypervisors	Delivered	Section 5.2.6
04	REST API as the public boundary	Delivered	Section 5.2.9
05	CLI client and single-line installation	Delivered	Section 5.3
06	Engineering quality and discipline	Delivered	Section 5.4 and Chapter 4
07	Groundwork for the pieces beyond this PoC	Partial	Section 6.2

Table 6.1. Status of the seven objectives at the close of the proof of concept.

are not implemented in this TFG. The architecture chapter and the design-decisions log document the extension points precisely enough that the work to come can proceed without revisiting the engine.

6.1.2 Main contributions

The contributions of this work are honest about the state of the art (Chapter 3). The PoC does not introduce fundamentally new techniques; it integrates known techniques in a specific combination that, to our knowledge, no other production system provides:

Independence from any single hypervisor. Every hypervisor sits behind a common connector contract, so VMware, Proxmox and future types (Hyper-V, KVM, cloud providers) can coexist and a new one is added by writing a single class, without touching the core. This generalizes the supervisor’s VMware-only starting point and frees an institution from vendor lock-in — it can mix hypervisors and migrate without rewriting. The mock connector proves the contract end-to-end; the real ones are the immediate next step.

Delegation of VM operation to non-technical users. A professor sets up a whole subject — a folder of machines per enrolled student — with a couple of commands, and the students operate their own machines without going through the administrator. The end user sees a domain-specific metaphor (*desks* and *computers*) that hides the hypervisor and the orchestrator’s own concepts alike. One person manages hundreds of VMs without effort that grows in proportion.

A working teaching application on a domain-agnostic engine. The teaching layer (the `teach` sub-commands) is a thin client of the generic core that turns a subject definition and a student list into the full per-student infrastructure. It is the practical proof that the two-layer split works: a complete domain workflow built on the engine without the engine knowing anything about teaching. The same seam (the REST API) would let a CTF organizer or a research-lab deployer reuse the core unchanged.

An inheritance model that scales the configuration. A tree of descriptors with cascade inheritance by data type lets the catalogue be defined once and reused across hundreds of near-identical nodes, and is the natural seat of a future permission model. A minor but uncommon detail of this model is the lexical-closure resolution of imported templates, borrowed from programming languages and applied to infrastructure definitions.

A proof of concept with full engineering discipline. The delivered code base, despite being a PoC, ships with 286 automated tests, branch coverage above 95% on the domain logic, strict static typing, continuous integration gates, a published image in the GitHub Container Registry and a tutorial for the operator. The discipline is itself a deliverable: the future iterations of the project — and a possible open-source product beyond the TFG — inherit a code base that is genuinely maintainable.

6.1.3 Known limitations

The limitations of the proof of concept are listed openly so that the assessment of the work is not based on optimistic assumptions.

- **No real hypervisor connector.** The mock connector proves the contract, but a deployment against a real VMware or Proxmox host is not implemented. The bottleneck is external: VPN access to the teaching VMware cluster was not available during the development period. The VMware connector is the immediate next task once access is granted.
- **Web interface limited to read-only.** A first version of the web GUI is delivered in Sprint 4 with the tree view, summary counters and per-descriptor detail; write operations (deploy, start, stop, undeploy, load) and the workstation/desk view for end users remain on the roadmap.
- **Teaching application is a first version.** The layer-2 teaching application is implemented (load a subject, generate the per-student folders and machines, export the student list) but minimal:

deploying a whole subject in one command, removing students who left the roster, and email notifications are designed and left as future work.

- **Authentication not implemented.** The RBAC model is fully designed (DEC-011, DEC-021, DEC-026) and only the superuser role is realistic in v1. The service assumes it runs in a trusted network. Authentication is part of the future-work list.
- **Best-effort persistence.** TinyDB does not support multi-document transactions. The cost of a crash in the middle of a multi-document write is accepted explicitly in DEC-030; the replacement by MongoDB lifts this limitation.
- **Single-host service.** The PoC assumes a single host running the daemon. Active/passive high availability is designed but not implemented.

6.2 Future work

The following extensions are the natural continuation of the work, ordered from the most immediate to the most ambitious. Most of them were anticipated in the design and have their extension points in place; they have been documented in `claude/ideas-pendientes.md` throughout the development.

6.2.1 Next sprints (immediate)

Real hypervisor connectors. The VMware connector is the highest priority, both because it unblocks an operational deployment against the existing teaching lab and because it validates the abstract connector contract against the messiest of the two target APIs (vSphere SOAP). The Proxmox connector follows the same plan once VMware is in.

Write operations in the web interface. The read-only v1 delivered in Sprint 4 covers tree navigation, summary counters and per-descriptor detail. The next iteration adds the write operations (deploy, start, stop, undeploy, load), the YAML editor and the workstation/desk view for end users.

Teaching application (layer 2). A single command takes a list of students and a base template and produces a complete subject deployment: one folder per student, one descriptor per student, the right cascade configuration. A second command recycles the whole subject at the end of the term.

Authentication and RBAC. Implement the role-based access control already designed. Users in a

YAML file in v1; an LDAP / Active Directory backend behind the `UserRepository` abstraction in v2. Session tokens persisted in the database, as designed in DEC-031.

6.2.2 Beyond the immediate roadmap

Richer web interface. Operational dashboards, advanced editing of the tree, alternative views from the hypervisor side.

IPAM integration. A real IPAM (e.g. phpIPAM, NetBox) replaces the internal IP-pool model. The `IPPoolRepository` is the seam.

Snapshot management. Snapshots through the orchestrator with inheritable policies (retention, automatic-snapshot triggers).

Datastores modelled as inheritable resources. The current proof of concept hardcodes the default datastore of each hypervisor. A first-class datastore model with alias-based references, similar to the existing template model, brings storage policy into the declarative layer.

Additional connectors. Hyper-V, KVM, public-cloud providers (AWS EC2, Azure VM). Each is a separate connector implementing the same abstract base class.

High availability. Active/standby deployment of the daemon, with MongoDB replacing TinyDB to provide multi-document transactions and replication. The session and job models were designed with this future in mind.

GitOps over VMs. The declarative YAML model lends itself to a Git-backed workflow: the tree of definitions lives in a Git repository, a push triggers `load`, and the orchestrator becomes the convergence engine. Equivalent to what ArgoCD or Flux do for Kubernetes, but for VM infrastructure.

Structured logging to syslog. The current logging is ad-hoc to standard output. A structured pipeline through the Python `logging` module to syslog is designed in DEC-023 and pending implementation.

Safety policies on destructive operations. `undeploy` currently succeeds on a running VM by stopping it first. A `--force` flag and a refuse-by-default policy on running VMs is part of the future safety work documented in `claude/ideas-pendientes.md`.

6.2.3 Beyond a proof of concept

The need that motivates this work is not exclusive to one school: any institution that runs virtual labs for non-expert users — other universities, vocational schools, research groups, corporate training — faces the same operational load and the same vendor lock-in. Early interest from outside the school (a research-centre contact and a secondary school migrating to Proxmox) suggests the value is real beyond the original case. A natural continuation beyond the TFG is to turn the proof of concept into an open-source product, with the several extension lines above as the road map and with room to explore sustainable open-source funding models. The intellectual-property and licensing questions that such a step raises are out of scope here and left for a later assessment.

The proof of concept proves the model. The path to a production system — and possibly to an open-source product — goes through the items above, in roughly the order listed. The engine is ready to receive them.

Appendix A.

Administrator's manual

This appendix is the operational manual for an administrator who installs and runs UnivOrch in an institutional environment. It does not repeat the conceptual material of Chapter 2; the reader is assumed to have followed the rest of the report. The manual covers requirements, installation, day-to-day operation, configuration of real hypervisors (designed but not implemented in the proof of concept), monitoring and maintenance.

The conventions are: a sequence of commands separated by line breaks is run in order; comments after `#` are not part of the command; placeholders in angle brackets (`<server>`) are to be replaced.

A.1 Requirements

The host that runs UnivOrch needs:

- Linux (any distribution with current packages).
- Docker Engine version 24 or newer.
- Docker Compose, version 2 (`docker compose`) or version 1 (`docker-compose`); the installer detects both.
- Outbound HTTPS to `ghcr.io` to pull the public image. If the host is in a closed network, the image can be mirrored to an internal registry; the configuration change is the registry name in `docker-compose.yml`.

The clients (CLI on a workstation, web browser on a laptop) need only network access to the daemon's port (8080 by default).

UnivOrch does not require root: a user in the `docker` group is enough.

A.2 Quick installation

The published installer takes care of everything for a single-host deployment. It downloads the `docker-compose.yml`, picks a free TCP port if 8080 is taken, pulls the image, creates the persistent volume and starts the container.

```
curl -sSL https://raw.githubusercontent.com/clamaveruma/univorch/main/install.sh | ba
```

A health check confirms the install:

```
curl -s http://localhost:8080/api/v1/health
# {"status":"ok"}
```

For a closer look at what the installer does before running it, substitute `bash` with `less` or `cat`:

```
curl -sSL https://raw.githubusercontent.com/clamaveruma/univorch/main/install.sh | le
```

A.3 Production installation

For production the recommended path is to clone the repository, edit `docker-compose.yml` to the local conventions and run `docker compose up -d`.

```
git clone https://github.com/clamaveruma/univorch.git
cd univorch
$EDITOR docker-compose.yml # adjust port, volume name, image tag
docker compose up -d
docker compose logs -f      # follow the daemon's log
```

The file `docker-compose.yml` exposes three pieces of configuration:

Image tag – pin the deployment to a specific version instead of `:latest`. Example: `ghcr.io/clamaveruma/univorch:v0.1.0`

The tags follow semantic versioning; `v0.x` indicates the API surface is not yet stable.

Port mapping – the public port the daemon listens on. The default is 8080 on the host mapped to 8080 in the container. Change the host side if the port is busy or if a reverse proxy fronts the daemon.

Named volume – the Docker named volume that contains the TinyDB JSON file. The default name is `univorch_data`; keep the default unless several UnivOrch instances coexist on the same host.

The image does *not* terminate TLS. In a production deployment, place the daemon behind a reverse proxy (nginx, Caddy, Traefik, the institutional load balancer) that handles HTTPS, forwards the request unchanged and trusts the daemon on HTTP. The daemon binds to all interfaces inside the container by default; restrict the binding through the proxy, not through UnivOrch.

A.4 Real hypervisor configuration (design)

The PoC ships only the mock connector. The real connectors (VMware vSphere and Proxmox VE) are designed but not implemented in v1. The configuration of a real hypervisor below describes the intended user interface; it will appear once the connectors are in code.

VMware ESXi / vSphere.

The administrator declares the hypervisor in the top administrative folder of the tree:

```
folders:
  admin:
    define hypervisors:
      esx1:
        type: vmware
        address: vcenter.dept.uma.es
        username: orchestrator@vsphere.local
        password: <stored encrypted>
        datacenter: TeachingLab
        datastores:
          fast: { id: SSD-Pool-1 }
          slow: { id: HDD-Pool-2 }
```

The base VMs available for cloning live in the same vSphere datacenter and have a snapshot ready for linked clones. The administrator prepares the base VMs once and refers to them by name

from templates.

Proxmox VE.

Proxmox uses an API token rather than a password:

```
folders:
  admin:
    define hypervisors:
      pve1:
        type: proxmox
        address: pve.dept.uma.es
        token_id: orchestrator@pve!api
        token_secret: <stored encrypted>
        node: pve-node-01
        datastores:
          local: { id: local-lvm }
```

The base VM is a template VM created in Proxmox; cloning is delegated to the standard Proxmox API.

Mock.

The mock connector requires no configuration. The administrator declares it the same way:

```
folders:
  admin:
    define hypervisors:
      mock01: { type: mock }
```

A.5 Loading definitions

Definitions are written in YAML and loaded into a folder of the tree with the `load` command. The destination can be any folder, not only the root; the YAML is relative to the destination (Section 2.7).

The administrator typically keeps the catalogue of definitions in a Git repository. A simple workflow is:

```
git clone git@servers.dept.uma.es:univorch-defs.git
cd univorch-defs
univorch load catalogue/hypervisors.yml /admin/
univorch load courses/networks-2026.yml /lab/
```

Two YAML files describe two complementary pieces of the tree: a catalogue of hypervisors and templates at /admin/, and a subject's worth of student descriptors at /lab/. The order of load calls matters when a later file references resources declared by an earlier one.

load is idempotent. Running the same file twice does not duplicate folders or descriptors; only new items are added. Properties of existing items are updated to match the file.

A.6 Day-to-day operation

The administrator interacts with the daemon through the CLI or the web GUI. The most frequent operations are listed below; ?? D gives the full reference.

Inspecting the tree.

The CLI is the fastest way to scan the current state. From inside the container:

```
docker exec -it univorch univorch
univorch /> tree /
univorch /> status /lab/networks/student01
univorch /> inspect /lab/networks/student01
```

The web GUI at <http://<server>:8080/> gives the same information graphically, with the summary counters at the top of the page.

Deploying and undeploying.

A descriptor in `provisioned` becomes a real VM with `deploy`; `undeploy` destroys the VM and returns the descriptor to `provisioned`.

```
univorch /> deploy /lab/networks/student01
univorch /> start /lab/networks/student01
univorch /> stop /lab/networks/student01
univorch /> undeploy /lab/networks/student01
```

For batch operations across a whole folder, scripting is the practical option in v1; the web GUI will offer the same operations on a click in the next sprint.

Recovering a broken descriptor.

A descriptor stuck in **broken** reports the cause in its Job history. Read the history, fix the underlying problem (restore credentials, free disk space, recover network connectivity to the hypervisor) and **force-undeploy** the descriptor to bring it back to **provisioned**:

```
univorch /> status /lab/networks/student09
univorch /> # ... read the job history ...
univorch /> force-undeploy /lab/networks/student09
```

The **force-undeploy** command is on the roadmap; until then, a broken descriptor is cleared by hand at the hypervisor (delete the residual VM if any) and a normal **undeploy** run from the CLI.

A.7 Monitoring and logs

The container writes the daemon's standard output to Docker's logging driver. Read the last hour with:

```
docker compose logs --since 1h -f
```

For a long-running system, the Docker logs should be forwarded to the institutional log collector (rsyslog, journald, ELK, Grafana Loki) using the standard **logging:** stanza of **docker-compose.yml**.

The orchestrator's *operation* log is the persisted Job history, not the standard output. The Job history is queried through the CLI or the web GUI and survives restarts. The default retention is 90 days; the value is configurable as a global property.

A.8 Backups and recovery

The full state of UnivOrch is a single JSON file in the named volume `univorch_data` (the database is TinyDB in v1, a choice that is replaced by MongoDB in production). A backup is a copy of the volume.

The simplest periodic backup is a wrapper around `docker cp`:

```
#!/bin/bash
set -e
ts=$(date +%F-%H%M)
docker cp univorch:/data/db.json /var/backups/univorch/univorch-$ts.json
```

The recommended retention is GFS (DEC-024): seven daily backups, four weekly backups, twelve monthly backups. A simple `cron` entry plus a small rotation script implements it.

Restoring is also a file copy in v1. Stop the daemon, replace the database file with the backup, start the daemon:

```
docker compose stop
docker cp /var/backups/univorch/univorch-2026-06-07-0300.json \
    univorch:/data/db.json
docker compose start
```

When the persistence layer moves to MongoDB, the same workflow is replaced by `mongodump` and `mongorestore` with the same GFS schedule.

A.9 Upgrades

Upgrading the daemon to a new minor or patch release is a tag change in `docker-compose.yml` followed by:

```
docker compose pull
docker compose up -d
```

Docker downloads the new image, stops the running container and starts a new one against the same persistent volume. The operational data is preserved.

Major upgrades (v1 → v2) may include a migration step on the database. The release notes of every major version describe the migration; until that path is established, the upgrade is a backup, a stop, an image swap, a migration command and a start.

A.10 Users and authentication

In v1 the daemon assumes the network it runs on is trusted: there is no authentication on the REST API and the only role enforced in code is the implicit *superuser*. The full RBAC design lives in Chapter 2 and is part of the immediate future work.

Until authentication is in place, the deployment should restrict network exposure: a reverse proxy that only accepts requests from the institutional VPN, a firewall rule that blocks the daemon's port to the outside, or both.

A.11 Troubleshooting

The container does not start.

`docker compose logs` usually shows the cause. The most common ones are a busy port (change the host port mapping), a permission problem on the volume (check the file owner) and a missing image (the network blocks `ghcr.io`; use the internal mirror).

The web GUI loads but the tree is empty.

The database is empty. Load a definition with `load` as described above.

Deploy fails immediately.

The connector cannot reach the hypervisor. Verify the hypervisor's address and credentials in the corresponding `define hypervisors:` block, and check the network from inside the container with `docker exec univorch curl -s <hypervisor>`.

A descriptor sits in unreachable.

The hypervisor was not reachable when an operation ran. The state is reactive; once communications are restored, an operation on that descriptor will move it back to a definite state. The Job history records the original communication error.

A.12 Where to find more information

- Source code: <https://github.com/clamaveruma/univorch>.
- Public image: ghcr.io/clamaveruma/univorch.
- REST API specification: <http://<server>:8080/api/v1/docs> (Swagger UI generated by FastAPI).
- Architecture and design: Chapter 5 of this report and `docs/architecture.md` in the repository.
- Design-decision log: `?? C` and `claude/decisiones.md` in the repository.

DRAFT

Appendix B. Detailed requirements analysis

This appendix reproduces the requirements analysis of UnivOrch v1 that was developed in Phase 2 of the project. The structured summary of the same analysis is in Section 5.1 of Chapter 5. The authoritative source lives at `docs/requirements.md` in the project repository; if the two diverge in the future, that document takes precedence.

B.1 Purpose and scope

The analysis covers the generic orchestrator core and the teaching application built on top. It records use cases, functional requirements, non-functional requirements, constraints and explicitly out-of-scope items. Architecture and technology choices do not appear here; they are deferred to Chapter 5 and Chapter 3 respectively.

B.2 Actors

Table 6.2 lists the actors of the system. A single person can hold different roles in different branches of the tree; the role lives on the folder, not on the user record (DEC-021).

B.3 Use cases

Use cases are grouped into three families: *core* use cases (generic orchestrator operations), *teaching* use cases (the layer-2 application) and *system* use cases (automatic behaviour). The catalogue contains 32 entries; the summary below lists every entry with its actor and one-sentence goal.

Actor	Description
Administrator (superuser)	Manages hypervisors, base templates, users, the full tree and global configuration.
Manager	A professor or equivalent. Manages assigned branches of the tree; uses both the core and the teaching application.
End user (student)	Operates only the VMs assigned to them through the desk metaphor (DEC-009).
Hypervisor	External system. UnivOrch talks to it through a connector; the hypervisor does not initiate actions.
System	UnivOrch itself, running the operation engine and the automatic tasks (backup rotation, retention cleanup, failure detection).

Table 6.2. *Actors of UnivOrch (detailed).*

B.3.1 Common

UC-AUTH-1 (Any user.) Authenticate against the system and obtain a session token.

B.3.2 Core — Administrator

UC-ADM-1 Manage users (create, edit, delete) through the web interface.

UC-ADM-2 Define hypervisors and their datastores with alias-based references.

UC-ADM-3 Define base templates (hardware parameters, base VM, datastore, source hypervisor).

UC-ADM-4 Create and organise folders, including optional IP pools that descendants inherit.

UC-ADM-5 Assign users to roles in any folder.

UC-ADM-6 Control what managers can access by deciding what is imported into each folder.

UC-ADM-7 Configure global properties such as Job retention.

UC-ADM-8 Force-undeploy a descriptor stuck in **broken** state.

UC-ADM-9 View the Job history of any branch.

UC-ADM-10 Restore the database from a backup (manual in v1).

UC-ADM-11 View operational reports (state of all VMs grouped by folder).

B.3.3 Core — Manager

UC-MGR-1 Create and organise folders inside the branches assigned to the manager, importing definitions from the parent.

UC-MGR-2 Create descriptors based on the templates allowed to the manager.

UC-MGR-3 Derive new templates from base templates; the manager sees the template definition but not the hypervisor's address or credentials (DEC-011).

UC-MGR-4 Assign students (and other managers) to roles in the manager's own folders.

UC-MGR-5 Trigger batch operations on a set of descriptors (deploy, undeploy, start, stop, pause, resume) and follow progress through a parent Job with child Jobs per VM.

UC-MGR-6 View the Job history restricted to the manager's scope.

UC-MGR-7 View a subject status report.

B.3.4 Teaching — Manager (layer 2)

UC-TEACH-1 Deploy a subject from a model desk and a student list: one desk per student, descriptors in provisioned, IPs assigned, notification emails sent, deployment report produced.

UC-TEACH-2 Update the student list of an existing subject; new students get desks, removed students are flagged according to policy.

UC-TEACH-3 Change a model VM; descriptors are updated and changes take effect on the next recreation (DEC-022).

B.3.5 Core — End user (student)

The student sees the desk metaphor (DEC-009) and never sees the tree.

UC-USR-1 View their own desks and the state of each VM.

UC-USR-2 Deploy a VM from `provisioned`.

UC-USR-3 Undeploy a VM with a double confirmation warning, because the action is irreversible.

UC-USR-4 Start, stop, pause or resume a deployed VM.

UC-USR-5 Recreate a VM (undeploy followed by deploy) as a way to recover from a corrupted system without administrator help.

UC-USR-6 View the status and connection information (typically the IP) of a VM, read from the hypervisor on demand.

B.3.6 System — automatic behaviour

UC-SYS-1 Run the operation engine: take operations from the queue, execute, record the result. Synchronous in v1; the Job model is designed for a future asynchronous queue (DEC-014, DEC-015).

UC-SYS-2 Periodically back up the database following a GFS retention policy (DEC-024).

UC-SYS-3 Remove Jobs older than the configured retention period (DEC-023).

UC-SYS-4 Detect an unreachable hypervisor reactively: when an operation fails by communication error, the Job is recorded as failed and the descriptor is marked `unreachable`.

B.4 Functional requirements

The functional requirements below are grouped by area. They restate the obligations that the use cases imply, in the form an engineer can check off during implementation.

B.4.1 Tree and folders

The system stores descriptors and folders in a hierarchical tree. A folder has a common definition that its children import. A parent folder *publishes* definitions; the child folder *imports* what it wants to be visible below. Folder creation is restricted by role (DEC-012).

B.4.2 Cascade inheritance

The effective definition of a descriptor combines all definitions from root to that node (DEC-010); each level may override the level above. Hypervisors, templates, datastores and role assignments are all inherited this way. The child folder chooses what to import from the parent, including the wildcard * (DEC-012). A user can read the full resolved definition of every imported item, but can only modify what is written at their own level. Hypervisor credentials and address are never visible to anyone below the administrator. There is no explicit *ownership* concept: it emerges from the role assignment (DEC-021).

B.4.3 Descriptors and states

A descriptor holds the VM definition and the reference to the real VM (the id used by the hypervisor) and tracks its own state. The four descriptor states are **provisioned**, **deployed** (with an optional **drifted** flag), **broken** and **unreachable** (DEC-022). Runtime states are queried on demand. Drift is detected reactively in v1.

B.4.4 Deployment and hypervisor connectors

deploy and **undeploy** are orchestrator-level operations; the connector exposes primitives only (DEC-016). Every connector implements **clone**, **delete**, **start**, **stop**, **force_stop**, **pause**, **resume**, **get_status** and **get_info**. **clone** takes a **mode** parameter; v1 only supports **linked**. v1 ships connectors for the mock hypervisor, VMware ESXi (designed) and Proxmox (designed). The orchestrator is non-invasive: the hypervisor keeps working if UnivOrch is stopped.

B.4.5 Jobs and the operation engine

Every operation creates a Job with a unique identifier and the lifecycle **pending** → **running** → **completed** or **failed** (DEC-014). Jobs cover VM operations, tree operations and permission changes. Batch operations spawn a parent Job and one child Job per item. Jobs are persisted in the database (DEC-015) and provide the user-facing history.

B.4.6 Users and access control

Users are stored in a YAML file managed by the administrator from the web interface (DEC-021). The three fixed roles are `superuser`, `manager` and `end_user` (DEC-011). Role assignment lives on the folder and inherits in cascade; it may be overridden in subfolders. The user store is behind an abstraction so it can later be replaced (LDAP, Active Directory).

B.4.7 IP address management

Each folder may define an IP pool: a contiguous range, mask and gateway (DEC-025). Pools cascade like every other property. At deploy time the system picks the next free address from the applicable pool; at undeploy time the address is released. How the VM ultimately receives the address (DHCP, cloud-init, etc.) is outside the scope of UnivOrch. External IPAM integration is not included in v1 (DEC-013).

B.4.8 Teaching application

The teaching application sits on top of the core and translates a teaching workflow into core operations (DEC-004). It deploys a subject from a model desk and a student list, applies the IP policy, sends notification emails and produces a deployment report. It supports updating the student list and changing the model VM.

B.4.9 Reports

v1 ships at least one operational report (VM state grouped by folder) and one teaching report (subject status for a manager). The full catalogue of possible reports is recorded for the future.

B.4.10 Interfaces

The system exposes a REST API used by external clients (DEC-018). A CLI works as both single commands and an interactive shell. A web interface covers all roles and is designed especially for the student. The CLI is a pure REST client of the daemon. The web interface is mounted on the daemon's HTTP server.

B.4.11 Persistence, logging and backup

Tree, descriptors, Jobs and users are persisted and survive restarts. Persistence goes through the Repository pattern (DEC-007). System logs use Python's standard `logging` (DEC-023). Operation logs are the Job history with configurable retention (default 90 days). The database is backed up periodically under a GFS retention policy (DEC-024); v1 restores are manual.

B.5 Non-functional requirements

Table 6.3 restates the non-functional requirements as soft targets that keep the implementation honest, not as contractual service levels.

Quality	Target
Usability	A non-technical user operates their VMs without any knowledge of hypervisors.
Security	Every operation requires authentication. Users only see and act on resources allowed by their role and folder assignments. Plain-text password storage is a known PoC limitation.
Performance	A realistic teaching load: hundreds of VMs across several subjects. Cascade inheritance keeps the configuration manageable at that scale.
Reliability	The system stays consistent across restarts. A failed operation always leaves a record in the Job history. An inconsistent descriptor goes to broken . The database can be restored from a backup.
Maintainability	New hypervisors are added by implementing the common connector interface, without touching the core. New applications are built on the core without modifying it (DEC-004).
Portability	The system runs as a Linux service. It does not require the hypervisors on the same host and does not interfere with their normal operation.

Table 6.3. Non-functional requirements of UnivOrch v1.

B.6 Constraints and assumptions

- The implementation language is Python; the version is chosen in Phase 4 (DEC-033).
- The system runs as a service on Linux.
- Hypervisors exist and are not part of UnivOrch; base VMs are created directly on the hypervisor by the administrator.
- Operations are synchronous in v1.
- Passwords are stored in plain text in v1 (PoC only).
- IP assignment uses simple per-folder pools in v1; external IPAM integration is out of scope (DEC-013, DEC-025).

B.7 Out of scope for v1

- Automatic reconciliation loop (changes are applied on demand) – DEC-006.
- High availability and clustering.
- External IPAM integration – DEC-013.
- Snapshot management through the orchestrator.
- Reference-only descriptors and discovery of pre-existing VMs – DEC-005b, DEC-020.
- LDAP / Active Directory integration – DEC-021.
- Text-mode user interface (TUI) – DEC-018.
- Web interface for backup restore – DEC-024.
- Per-folder fine-grained permission customisation – DEC-011.

Appendix C. Summary of technical decisions

This appendix summarises the 36 technical decisions recorded during the project. Each row lists the identifier, a one-line title, the phase in which the decision was taken and the chapter of this report where the decision is discussed in context. The full rationale of every decision (motivations, alternatives considered, traceability between decisions) lives in the project repository, in `claude/decisions.md`.

C.1 Summary table

ID	Title	Phase	Chapter
DEC-001	Markdown for persistent memory	1	3
DEC-002	Context-directory structure (<code>claude/</code>)	1	3
DEC-003	Journal as primary source of truth	1	3
DEC-004	Two-layer architecture (engine + domain app)	1	2
DEC-005	Terminology: "descriptor"	1	2
DEC-005b	Descriptor types (normal vs reference)	2	2
DEC-006	Declarative architecture	1	2
DEC-007	Repository pattern for persistence	3	5
DEC-008	Use of design patterns (pragmatic)	1	5
DEC-009	End-user metaphor (<i>desk / computer</i>)	1	2
DEC-010	Cascade inheritance required from v1	1	2
DEC-011	RBAC with hierarchical inheritance, three roles	1	2
DEC-012	Explicit imports + lexical closure of templates	2	2
DEC-013	IP management out of v1	1	5
DEC-014	Operations model: Job pattern	1	5

ID	Title	Phase	Chapter
DEC-015	Jobs persisted in the database	1	5
DEC-016	Hypervisor connector operations	2	5
DEC-017	Datastores as aliased resources	1	5
DEC-018	Client interfaces (CLI + Web GUI v1)	1	5
DEC-019	Project name (UnivOrch)	1	1
DEC-020	Access to pre-existing VMs (out of v1)	1	5
DEC-021	User management (YAML in v1; LDAP future)	2	5
DEC-022	Descriptor state machine (4 states + drift flag)	2	2
DEC-023	Logs and operation retention	2	5
DEC-024	Database backup policy (GFS)	2	5
DEC-025	IP management: pools per folder	2	5
DEC-026	Inheritance rule by data type	3	2
DEC-027	Declarative load/plan with fail-fast validation	3	5
DEC-028	Job engine: Command pattern, locking, sync v1	3	5
DEC-029	Connectors as ABCs + registry	3	5
DEC-030	TinyDB → MongoDB; repositories per aggregate	3	5
DEC-031	Facade and centralised RBAC	3	5
DEC-032	Refined definition of broken state	3	5
DEC-033	Technology stack v1	4	3
DEC-034	Bounded adoption of Pydantic v2	4	5
DEC-035	Operation idempotency (in the orchestrator)	4	5
DEC-036	Connector type registry + live-session pool	6	5

C.2 Structural decisions, in detail

Six of the thirty-six decisions carry more weight than the rest because they shape the architecture as a whole. They are presented in full below; the references in parentheses point to the chapter section where each is exercised.

DEC-004 — Two-layer architecture (Section 2.6).

The system is split into a domain-agnostic engine (folders, descriptors, jobs, inheritance, connectors) and a domain-specific application that translates the vocabulary of a use case (teaching, CTF, research) into operations of the engine. The boundary is the public REST API of the daemon. The decision is the conceptual root of the entire project: it is what lets the same engine support multiple domain applications.

DEC-026 — Inheritance rule by data type (Section 2.5).

Cascade inheritance applies a different merge rule depending on the data type of the field. Scalars overwrite, lists accumulate, dictionaries merge recursively. Specific fields that do not fit the default (the IP pool, for example) declare an explicit override. The rule is uniform across configuration properties *and* role assignments; the same code resolves both.

DEC-028 — Job engine: Command pattern, locking, synchronous v1.

Every state-changing operation is encapsulated in a Command class. The Command separates validation (returns the list of errors without touching state) from execution (mutates state and may raise). The engine wraps every Command in a Job that is persisted before execution. The proof of concept executes Commands synchronously; the architecture allows asynchronous execution with locking without changes to the engine surface.

DEC-029 — Hypervisor connectors as ABCs (Section 5.2.6).

The contract between the engine and a hypervisor is defined as an abstract base class with a small set of life-cycle operations (`clone`, `delete`, `start`, `stop`, `get_status` and a few more). The mock connector that ships with v1 implements the same contract. A new hypervisor (VMware, Proxmox) is added by writing a new concrete subclass; the engine itself is not touched.

DEC-031 — Facade and centralised RBAC (Section 5.2.6).

The `OrchestratorService` class is the single entry point of the engine and the only place where RBAC checks happen. Every client (CLI, Web GUI, future teaching application) routes through the facade; there is no shortcut.

DEC-036 — Connector type registry + live-session pool.

The engine keeps two distinct structures. A registry of connector *types* maps a type name (`mock`, `vmware`, `proxmox`) to its connector class; this is what is patched when a new hypervisor support is added. A pool of *live sessions* keeps the open connections to the configured hypervisors; this is what survives a request and is reused across operations. The decision was added late in the project after the architecture diagrams revealed that the two concepts had been conflated in the implementation.

Appendix D. CLI reference

This appendix documents the `univorch` command-line client. The CLI is a pure HTTP client of the daemon: every command translates into one or more calls to the REST API and prints the result. Inside the production Docker container the binary is on the `PATH`; on a development machine it is available with `uv run univorch`.

D.1 Invocation modes

The CLI supports the same set of commands in two modes:

Single-shot (bash mode) – one command per invocation, suitable for shell scripts. Example: `univorch deploy /lab/networks/student01`.

Interactive REPL – launched with no arguments. Carries a prompt that shows the current folder and (optionally) the remote daemon. Supports history, tab-completion of paths and in-shell `help <command>`.

D.2 Selecting the daemon

The CLI must know where the daemon listens. The selection follows this precedence (first match wins):

1. The `--remote URL` argument on the command line.
2. The `UNIVORCH_REMOTE` environment variable.
3. The built-in default `http://localhost:8080`.

Inside the REPL, the `connect <URL>` command switches the target daemon for the rest of the session and updates the prompt to show the new target.

D.3 Command reference

Table 6.5 lists the implemented commands at the close of Sprint 4. The detailed description of each command follows.

Command	Purpose
<code>cd [path]</code>	Change the current folder in the REPL.
<code>pwd</code>	Print the current folder.
<code>ls [path]</code>	List one level of a folder (alias of <code>list</code>).
<code>list [path]</code>	Same as <code>ls</code> .
<code>tree [path] [-d]</code>	Print the full subtree under a folder.
<code>inspect <path> [-l]</code>	Show the resolved (or local-only) definition.
<code>status <path></code>	Show the descriptor state and the runtime state.
<code>deploy <path></code>	Move a descriptor from <code>provisioned</code> to <code>deployed</code> .
<code>undeploy <path></code>	Move a <code>deployed</code> descriptor back to <code>provisioned</code> .
<code>start <path></code>	Power on a deployed VM.
<code>stop <path></code>	Power off a deployed VM.
<code>load <file> [dest]</code>	Load a YAML definition into a folder.
<code>connect <url></code>	(REPL only) switch the live target daemon.
<code>help [cmd]</code>	Built-in help; one command without args lists all.
<code>quit</code>	(REPL only) leave the shell.

Table 6.5. Commands provided by the `univorch` CLI.

D.3.1 Navigation: `cd`, `pwd`

`cd [path]` changes the current folder of the REPL. The path follows the conventions of a Unix shell: it may be absolute or relative, `.` and `..` are honoured, and `..` at the root stays at the root. With no argument, `cd` does nothing; `pwd` prints the current folder.

D.3.2 Listing: `ls`, `list`, `tree`

`ls` (alias `list`) prints one level of children of a folder, with a glyph that marks the state of each descriptor. `tree` prints the entire subtree under a folder. The flag `-d` (or `--folders-only`) of `tree` hides descriptors and shows only the folder structure, the equivalent of the Linux `tree -d`.

The state glyphs used by both commands match the iconography of the web GUI: an empty square for `provisioned`, a filled square for `deployed`, a cross for `broken` and a warning triangle for `unreachable`.

D.3.3 Inspection: `inspect`, `status`

`inspect <path>` shows the entity at a tree path. For descriptors, the default mode merges the template, the cascade inheritance and the lexical closure, so the displayed definition is the *effective* one. With `-l` (or `--local`), only the fields explicitly written at the node are shown.

`status <path>` reports both axes of the descriptor's state: the orchestrator's state (`provisioned`, `deployed`, `broken`, `unreachable`) and, when applicable, the runtime state read from the hypervisor (`running`, `stopped`, `paused`).

D.3.4 Lifecycle: `deploy`, `undeploy`, `start`, `stop`

The four lifecycle commands take a descriptor path and submit the matching operation to the daemon. The daemon performs the action through the connector that the descriptor's hypervisor declares.

`deploy` moves the descriptor from `provisioned` to `deployed`. `undeploy` moves it back to `provisioned` and removes the VM from the hypervisor. `start` and `stop` change the runtime state of an already deployed VM. All four are idempotent: ordering an action that is already satisfied returns success with the message *"no change"* (DEC-035).

D.3.5 Loading definitions: `load`

`load <file> [dest]` reads a YAML definition file and materialises it inside the destination folder. The file describes the folders and descriptors to add; the destination is the parent. Without a destination, the current folder is used. The YAML is relative to the destination, which is what makes a definition file portable across installations.

The load operation is fail-fast: if any item fails validation, the whole load is rejected and the tree is not modified.

D.3.6 REPL-only commands: **connect**, **help**, **quit**

connect <url> switches the live session to a different daemon. The prompt updates to show the new target between square brackets (for example `univorch [http://aulario.uma.es:8080] /lab/>`). The change is not persisted; the next REPL session goes back to the precedence rule above.

help alone lists the commands; **help** <cmd> prints the argparse-generated help for a specific command, with the same syntax and examples shown above. **quit** (or Ctrl+D on an empty line) leaves the shell.

D.4 Examples

A typical session inside the production container:

```
$ docker exec -it univorch univorch
univorch /> load /opt/univorch/examples/setup.yml /
6 items applied.
univorch /> cd /lab/networks
univorch /lab/networks/> ls
[ ] student01
[ ] student02
[ ] student03
univorch /lab/networks/> deploy student01
deploy /lab/networks/student01: ok (5.6s)
univorch /lab/networks/> status student01
descriptor state : deployed
runtime state    : stopped
vm id            : mock-7f3a91
univorch /lab/networks/> start student01
start /lab/networks/student01: ok
```

```
univorch /lab/networks/> quit
```

The same operations from a shell script:

```
#!/bin/bash
export UNIVORCH_REMOTE=http://orquestador.dept.uma.es:8080
univorch load /opt/univorch/examples/setup.yml /
for s in student01 student02 student03; do
    univorch deploy /lab/networks/$s
done
```

The CLI surface is intentionally small: the same vocabulary appears in the YAML model, in the REST API, in the CLI and in the web GUI. Adding new commands in the future means adding new operations to the orchestrator's facade and exposing them; the CLI's argparse boilerplate is the smaller part of the work.

DRAFT

Appendix E.

Conclusiones y líneas futuras (castellano)

Este apéndice reproduce en castellano el contenido del Chapter 6, según exige la normativa de la EI UMA cuando la memoria del TFG se redacta en inglés. La estructura es la misma que la del capítulo original.

E.1 Conclusiones

E.1.1 Cumplimiento de los objetivos

Los siete objetivos declarados en Section 1.2 se revisan aquí frente a lo realmente entregado. La Table 6.6 resume la valoración; los párrafos posteriores añaden las matizaciones que la tabla no puede recoger.

Los objetivos O1 a O6 se cumplen con el sistema entregado: el motor funciona, el modelo declarativo con herencia en cascada y cierre léxico de plantillas opera de extremo a extremo, el contrato del conector está definido y ejercitado por la implementación de prueba, la API REST expone todas las operaciones públicas, la CLI es un cliente HTTP puro de esa misma API, y las prácticas de ingeniería (desarrollo dirigido por pruebas, tipado estático estricto, integración continua, distribución empaquetada) se mantienen en todo el proyecto.

El objetivo O7 se marca como *parcial*: todos los puntos de extensión están diseñados y la mayoría conectados en el código, pero los siguientes entregables que dependen de ellos (conectores reales, interfaz web, aplicación docente) no se implementan en este TFG. El capítulo de arquitectura y el registro de decisiones de diseño documentan los puntos de extensión con la precisión necesaria para que el trabajo posterior pueda continuar sin visitar el motor.

ID	Objetivo	Estado	Referencia
01	Núcleo de orquestación agnóstico del hipervisor	Cumplido	Sections 5.2 and 5.3
02	Modelo declarativo con herencia	Cumplido	Sections 5.2.5 and 5.2.7
03	Contrato de conector para nuevos hipervisores	Cumplido	Section 5.2.6
04	API REST como frontera pública	Cumplido	Section 5.2.9
05	Cliente CLI e instalación en una línea	Cumplido	Section 5.3
06	Calidad y disciplina de ingeniería	Cumplido	Section 5.4 and Chapter 4
07	Bases para las piezas posteriores a la PoC	Parcial	Section E.2

Table 6.6. Estado de los siete objetivos al cierre de la prueba de concepto.

E.1.2 Aportaciones principales

Las aportaciones de este trabajo son honestas respecto al estado del arte (Chapter 3). La prueba de concepto no introduce técnicas fundamentalmente nuevas; integra técnicas conocidas en una combinación específica que, hasta donde sabemos, ningún otro sistema en producción ofrece:

Independencia de cualquier hipervisor concreto. Cada hipervisor queda tras un contrato de conector común, de modo que VMware, Proxmox y tipos futuros (Hyper-V, KVM, proveedores cloud) pueden coexistir y añadir uno nuevo es escribir una sola clase, sin tocar el núcleo. Esto generaliza el punto de partida (las librerías del tutor, solo VMware) y libera a la institución de la dependencia de un proveedor: puede mezclar hipervisores y migrar sin reescribir. El conector de prueba (mock) demuestra el contrato de extremo a extremo; los reales son el siguiente paso inmediato.

Delegación de la operación de VMs a usuarios no técnicos. Un profesor monta una asignatura entera — una carpeta de máquinas por cada alumno matriculado — con un par de comandos, y los alumnos operan sus máquinas sin pasar por el administrador. El usuario final ve una metáfora específica del dominio (*mesas y ordenadores*) que oculta tanto el hipervisor como los propios conceptos del orquestador. Una persona gestiona cientos de VMs sin un esfuerzo que crezca en proporción.

Una aplicación docente funcional sobre un motor agnóstico. La capa docente (los subcomandos

teach) es un cliente fino del motor genérico que convierte la definición de una asignatura y una lista de alumnos en toda la infraestructura por alumno. Es la prueba práctica de que la separación en dos capas funciona: un flujo de dominio completo construido sobre el motor sin que el motor sepa nada de docencia. La misma costura (la API REST) permitiría que un organizador de CTF o un desplegador de laboratorios de investigación reutilizaran el núcleo sin cambios.

Un modelo de herencia que escala la configuración. Un árbol de descriptores con herencia en cascada por tipo de dato permite definir el catálogo una vez y reutilizarlo en cientos de nodos casi idénticos, y es el asiento natural de un futuro modelo de permisos. Un detalle menor pero poco común de este modelo es la resolución por cierre léxico de las plantillas importadas, tomada de los lenguajes de programación y aplicada a definiciones de infraestructura.

Prueba de concepto con disciplina completa de ingeniería. El código entregado, a pesar de ser una PoC, se acompaña de 286 pruebas automáticas, cobertura de ramas superior al 95% en la lógica de dominio, tipado estático estricto, controles de calidad en integración continua, una imagen publicada en el GitHub Container Registry y un tutorial para el operador. La disciplina es en sí misma un entregable: las iteraciones futuras del proyecto — y un posible producto de código abierto más allá del TFG — heredan un código realmente mantenible.

E.1.3 Limitaciones conocidas

Las limitaciones de la prueba de concepto se enumeran abiertamente para que la valoración del trabajo no se base en supuestos optimistas.

- **Sin conector real de hipervisor.** El conector de prueba (mock) demuestra el contrato, pero un despliegue contra un host real de VMware o Proxmox no está implementado. El cuello de botella es externo: el acceso VPN al cluster VMware docente no estuvo disponible durante el periodo de desarrollo. El conector VMware es la siguiente tarea inmediata cuando se conceda el acceso.
- **Interfaz web limitada a solo lectura.** En el Sprint 4 se entrega una primera versión de la interfaz web con la vista del árbol, los contadores resumen y el detalle por descriptor; las operaciones de escritura (deploy, start, stop, undeploy, load) y la vista de mesa/ordenador para los usuarios finales quedan pendientes para la siguiente iteración.

- **Aplicación docente en primera versión.** La aplicación docente de capa 2 está implementada (cargar una asignatura, generar las carpetas y máquinas por alumno, exportar la lista de alumnos) pero es mínima: el despliegue de una asignatura entera en un comando, la baja de los alumnos que dejan la lista y los avisos por correo quedan diseñados y como trabajo futuro.
- **Autenticación no implementada.** El modelo de RBAC está completamente diseñado (DEC-011, DEC-021, DEC-026) y solo el rol de superusuario es realista en la v1. El servicio asume que se ejecuta en una red de confianza. La autenticación forma parte de la lista de trabajo futuro.
- **Persistencia best-effort.** TinyDB no soporta transacciones multidocumento. El coste de una caída en mitad de una escritura multidocumento se acepta explícitamente en DEC-030; la sustitución por MongoDB elimina esta limitación.
- **Servicio en un solo host.** La PoC asume un único host ejecutando el demonio. La alta disponibilidad activo/pasivo está diseñada pero no implementada.

E.2 Trabajo futuro

Las siguientes extensiones son la continuación natural del trabajo, ordenadas desde las más inmediatas a las más ambiciosas. La mayoría estaban anticipadas en el diseño y tienen sus puntos de extensión preparados; se han documentado en `claude/ideas-pendientes.md` a lo largo del desarrollo.

E.2.1 Sigüientes sprints (inmediato)

Conectores reales de hipervisor. El conector VMware es la máxima prioridad, tanto porque desbloquea un despliegue operativo contra el laboratorio docente existente como porque valida el contrato abstracto del conector contra la API más compleja de las dos previstas (vSphere SOAP). El conector Proxmox sigue el mismo plan una vez que VMware esté en marcha.

Operaciones de escritura en la interfaz web. La versión de solo lectura entregada en el Sprint 4 cubre la navegación del árbol, los contadores resumen y el detalle por descriptor. La siguiente iteración añade las operaciones de escritura (deploy, start, stop, undeploy, load), el editor YAML y la vista de mesa/ordenador para los usuarios finales.

Aplicación docente (capa 2). Un solo comando toma una lista de alumnos y una plantilla base y

produce un despliegue completo de la asignatura: una carpeta por alumno, un descriptor por alumno, la configuración de cascada correcta. Un segundo comando recicla toda la asignatura al final del curso.

Autenticación y RBAC. Implementar el control de acceso basado en roles ya diseñado. Usuarios en un fichero YAML en la v1; un backend LDAP / Active Directory detrás de la abstracción `UserRepository` en la v2. Tokens de sesión persistidos en la base de datos, tal y como prevé DEC-031.

E.2.2 Más allá del horizonte inmediato

Interfaz web más rica. Cuadros de mando operativos, edición avanzada del árbol, vistas alternativas desde el lado del hipervisor.

Integración con IPAM. Un IPAM real (por ejemplo `phpIPAM`, `NetBox`) reemplaza al modelo interno de *pools* de IPs. El `IPPoolRepository` es el punto de costura.

Gestión de snapshots. Snapshots a través del orquestador con políticas heredables (retención, disparadores automáticos de snapshot).

Datastores modelados como recursos heredables. La prueba de concepto actual codifica el datastore por defecto de cada hipervisor. Un modelo de datastore de primera clase con referencias por alias, similar al modelo de plantillas existente, incorpora la política de almacenamiento a la capa declarativa.

Conectores adicionales. Hyper-V, KVM, proveedores de nube pública (AWS EC2, Azure VM). Cada uno es un conector independiente que implementa la misma clase base abstracta.

Alta disponibilidad. Despliegue activo/pasivo del demonio, con MongoDB sustituyendo a TinyDB para proporcionar transacciones multidocumento y replicación. Los modelos de sesión y Jobs se diseñaron con este futuro en mente.

GitOps sobre VMs. El modelo declarativo YAML se presta a un flujo de trabajo respaldado por Git: el árbol de definiciones vive en un repositorio Git, un *push* dispara *load* y el orquestador se convierte en el motor de convergencia. Equivalente a lo que hacen ArgoCD o Flux para Kubernetes, pero para infraestructura de VMs.

Logging estructurado a syslog. El registro actual es *ad-hoc* a la salida estándar. Una canalización estructurada mediante el módulo `logging` de Python hacia syslog está diseñada en DEC-023 y pendiente de implementar.

Políticas de seguridad en operaciones destructivas. `undeploy` actualmente tiene éxito sobre una VM en ejecución parándola antes. Un *flag* `--force` y una política de rechazo por defecto sobre VMs en ejecución forman parte del trabajo de seguridad futuro documentado en `claude/ideas-pendientes.md`.

E.2.3 Más allá de una prueba de concepto

La necesidad que motiva este trabajo no es exclusiva de una escuela: cualquier institución que ofrezca laboratorios virtuales a usuarios no expertos — otras universidades, centros de formación profesional, grupos de investigación, formación corporativa — afronta la misma carga operativa y la misma dependencia de un proveedor. El interés temprano desde fuera de la escuela (un contacto de un centro de investigación y un instituto que migra a Proxmox) sugiere que el valor es real más allá del caso original. Una continuación natural más allá del TFG es convertir la prueba de concepto en un producto de código abierto, con las líneas de ampliación anteriores como hoja de ruta y con margen para explorar modelos de financiación sostenibles para código abierto. Las cuestiones de propiedad intelectual y licencias que ese paso plantea quedan fuera del alcance de esta memoria y se dejan para una valoración posterior.

La prueba de concepto demuestra el modelo. El camino hasta un sistema en producción — y posiblemente hasta un producto de código abierto — pasa por los puntos anteriores, aproximadamente en el orden listado. El motor está preparado para recibirlos.

Bibliography

- [1] Harold Abelson, Gerald J. Sussman, and Julie Sussman. *Structure and Interpretation of Computer Programs*. 2nd. MIT Press, 1996. ISBN: 0-262-01153-0.
- [2] Matthias Bolte et al. "Non-intrusive virtualization management using libvirt". In: *Proc. Design, Automation & Test in Europe (DATE)*. IEEE, 2010, pp. 574–579. DOI: [10.1109/DATE.2010.5457142](https://doi.org/10.1109/DATE.2010.5457142).
- [3] Mark Burgess. "On the theory of system administration". In: *Science of Computer Programming* 49.1–3 (2003), pp. 1–46. DOI: [10.1016/j.scico.2003.08.001](https://doi.org/10.1016/j.scico.2003.08.001).
- [4] Crossplane Authors. *Crossplane Documentation: Composite Resource Definitions and Compositions*. 2024. URL: <https://docs.crossplane.io/latest/concepts/composite-resource-definitions/>.
- [5] Distributed Management Task Force. *Open Virtualization Format (OVF) Specification, DSP0243*. 2024. URL: <https://www.dmtf.org/standards/ovf>.
- [6] Cynthia E. Irvine et al. "Labtainers: A Docker-based Framework for Cybersecurity Labs". In: *USENIX Workshop on Advances in Security Education (ASE)*. 2017. URL: https://www.usenix.org/system/files/conference/ase17/ase17_paper_irvine.pdf.
- [7] Kubernetes SIG-Multitenancy. *Introducing Hierarchical Namespaces*. 2020. URL: <https://kubernetes.io/blog/2020/08/14/introducing-hierarchical-namespaces/>.
- [8] OpenNebula Systems. *OpenNebula 6.10 Documentation: Users, Groups and Resource Management*. 2026. URL: https://docs.opennebula.io/6.10/management_and_operations/users_groups_management/.
- [9] oVirt Project. *oVirt Administration Guide: Data Centers, Clusters and Templates*. 2024. URL: https://www.ovirt.org/documentation/administration_guide/.
- [10] Akond Rahman, Rezvan Mahdavi-Hezaveh, and Laurie Williams. "A systematic mapping study of infrastructure as code research". In: *Information and Software Technology* 108 (2019), pp. 65–77. DOI: [10.1016/j.infsof.2018.12.004](https://doi.org/10.1016/j.infsof.2018.12.004).

- [11] Sebastián Ramírez. *FastAPI documentation*. 2026. URL: <https://fastapi.tiangolo.com/>.
- [12] Red Hat. *Virtual Machines as Code with OpenShift GitOps and OpenShift Virtualization*. 2024. URL: <https://www.redhat.com/en/blog/virtual-machines-as-code-with-openshift-gitops-and-openshift-virtualization>.
- [13] Ravi S. Sandhu et al. "Role-Based Access Control Models". In: *Computer* 29.2 (1996), pp. 38–47. DOI: [10.1109/2.485845](https://doi.org/10.1109/2.485845).
- [14] The Apache Software Foundation. *Apache CloudStack 4.22 Administration Guide: Roles, Accounts, Users, and Domains*. 2026. URL: <https://docs.cloudstack.apache.org/en/latest/adminguide/accounts.html>.
- [15] Jan Vykopal et al. "KYPO Cyber Range: Design and Use Cases". In: *Proc. 12th International Conference on Software Technologies (ICSOFT)*. 2017. DOI: [10.5220/0006428203100321](https://doi.org/10.5220/0006428203100321).
- [16] Christian Willems and Christoph Meinel. "Online assessment for hands-on cyber security training in a virtual lab". In: (2012). DOI: [10.1109/EDUCON.2012.6201149](https://doi.org/10.1109/EDUCON.2012.6201149).

DRAFT



UNIVERSIDAD
DE MÁLAGA

| **uma.es**

ETS. de Ingeniería Informática

Bulevar Louis Pasteur, 35

Campus de Teatinos

29071 Málaga